Paul E. McKenney, Meta Platforms Kernel Team

Kernel Recipes, September 24, 2024



Case Study: Concurrent Counting

As Easy as 1, 2, 3!

https://mirrors.edge.kernel.org/pub/linux/kernel/people/paulmck/perfbook/perfbook.html Chapter 5

© 2024 Meta Platforms

Recette Pour le Comptage Simultané

- Une pincée de connaissance des lois de la physique
- Compréhension modeste du matériel informatique
- Compréhension approfondie des exigences
- Conception soignée, y compris la synchronisation
- Validation brutale

https://mirrors.edge.kernel.org/pub/linux/kernel/people/paulmck/perfbook/perfbook.html Chapitre 3

Recette Pour le Comptage Simultané

- Une pincée de connaissance des lois de la physique
- Compréhension modeste du matériel informatique
- Compréhension approfondie des exigences
- Conception soignée, y compris la synchronisation
- Validation brutale

https://mirrors.edge.kernel.org/pub/linux/kernel/people/paulmck/perfbook/perfbook.html Chapitre 3

Recipe for Concurrent Counting

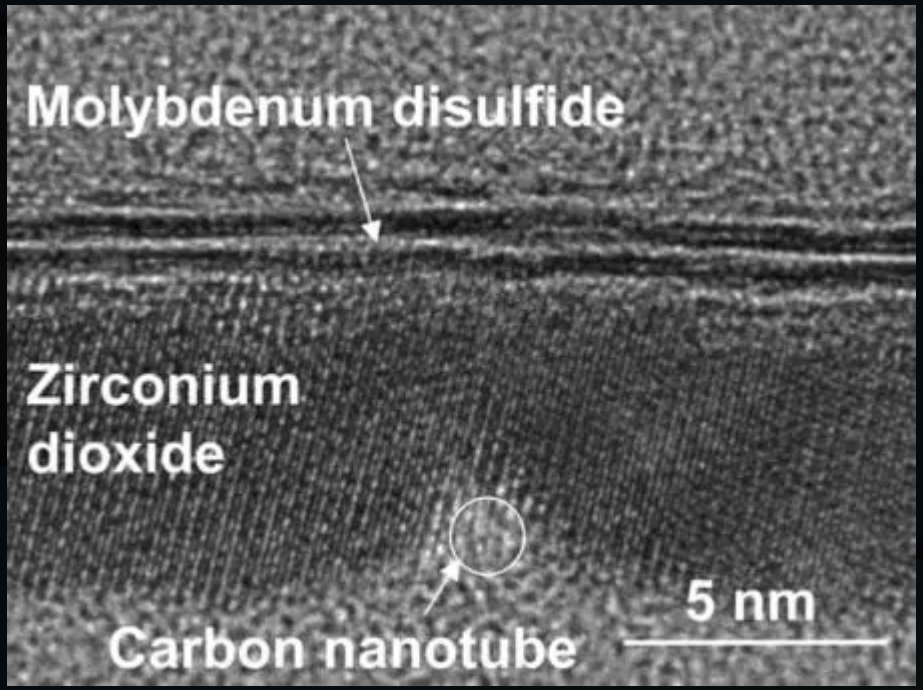
- **One pinch knowledge of laws of physics**
- **Modest understanding of hardware**
- **Thorough understanding of requirements**
- Careful design, including synchronization
- Brutal validation

https://mirrors.edge.kernel.org/pub/linux/kernel/people/paulmck/perfbook/perfbook.html Chapter 3

Distribution

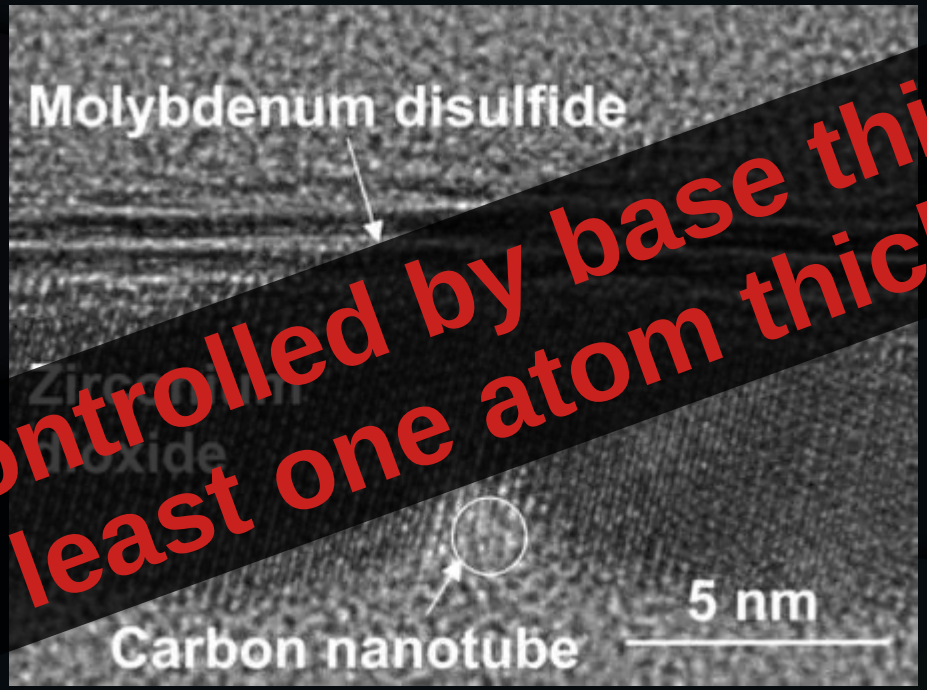
What Laws of Physics???

Laws of Physics: Atoms Are Too Big!!!



Each spot is an atom. Qingxiao Wang/UT Dallas ca. 2016.

Laws of Physics: Atoms Are Too Big!!!



Speed controlled by base thickness:
At least one atom thick!!!

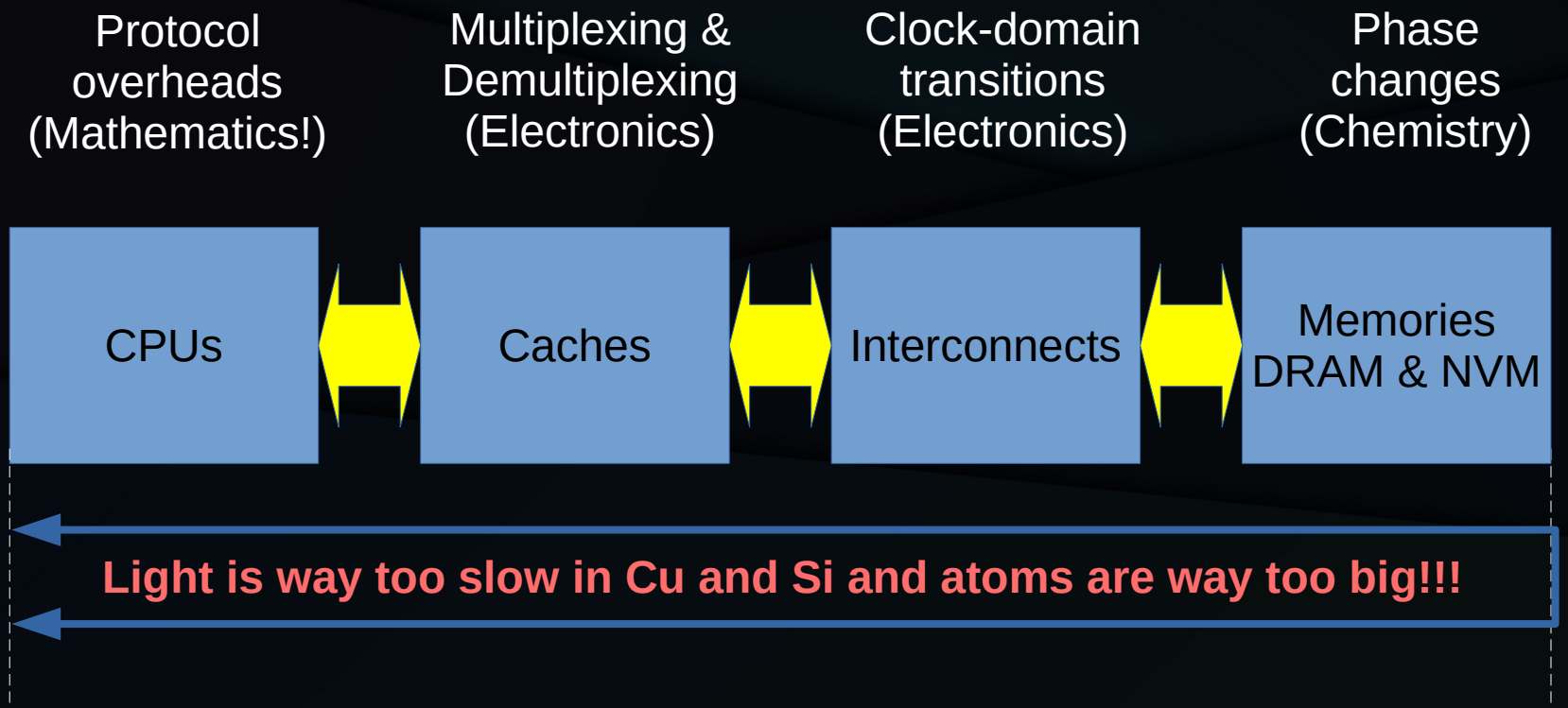
Each spot is an atom. Qingxiao Wang/UT Dallas ca. 2016.

Laws of Physics: Light Is Too Slow!!!

- Following the footsteps of Admiral Hopper:
  - Light goes 11.803 inches/ns in a vacuum
    - Or, if you prefer, 1.0097 lengths of A4 paper per nanosecond
    - Light goes 1 ***width*** of A4 paper per nanosecond in 50% sugar solution
  - But over and back: 5.9015 in/ns
  - But not 1GHz! Instead, ~2GHz: ~3in/ns
  - But Cu: ~1 in/ns, or Si transistors: ~0.1 in/ns
  - Plus other slowdowns: prototols, electronics, ...

“One nanosecond per foot” courtesy of Grace Hopper (https://www.youtube.com/watch?v=9eyFDBPk4Yw)
https://en.wikipedia.org/wiki/List_of_refractive_indices A 50% sugar solution is “light syrup”.

Laws of Physics: Data Is Slower!!!



Laws of Physics: Summary

- The speed of light is finite (especially in Cu and Si) and atoms are of non-zero size
- Mathematics, electronics, and chemistry also take their toll
- Systems are fast, so this matters

“Gentlemen, you have two fundamental problems:
(1) the finite speed of light and (2) the atomic nature of matter.” *

* Gordon Moore quoting Stephen Hawking

Laws of Physics: Summary

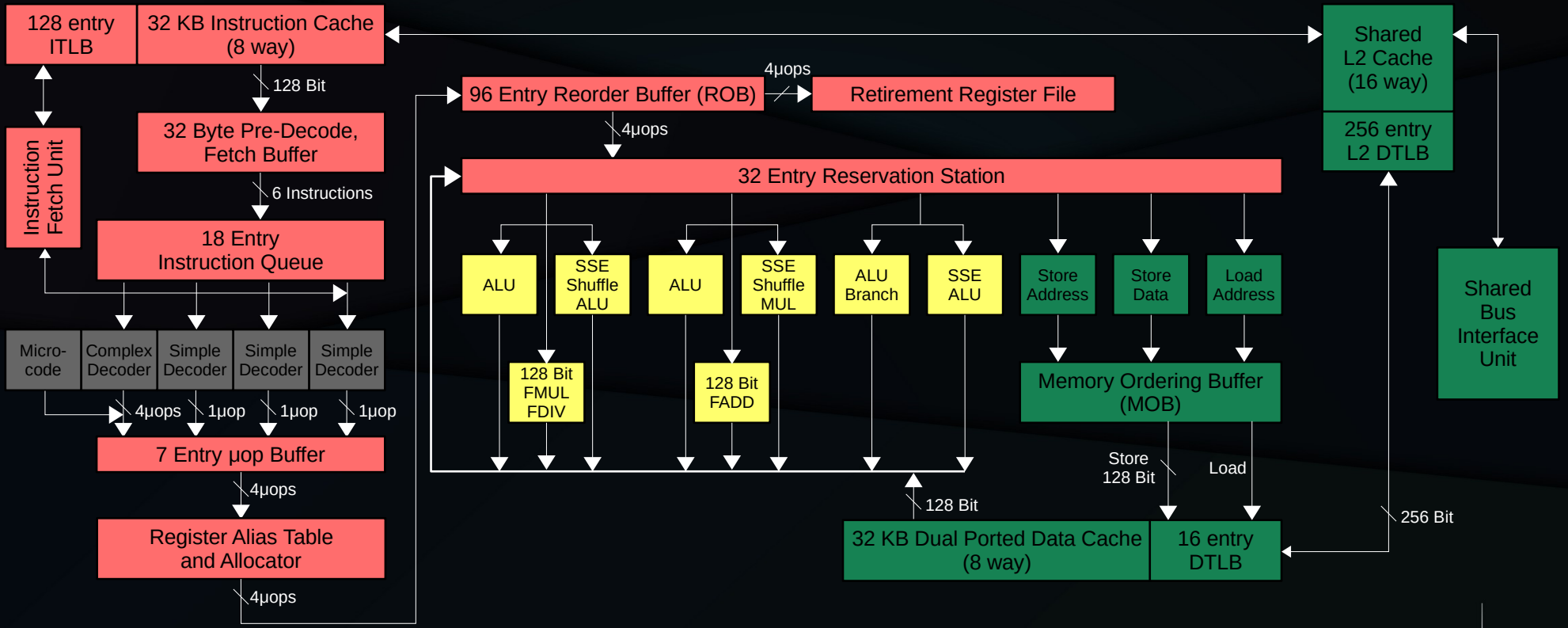
- The speed of light is finite (especially in Cu and Si) and atoms are of non-zero size
- Mathematics, electronics, and chemistry also take their toll
- Systems are fast, so this matters

Hardware architects are well aware of all this

"Gentlemen, you have two fundamental problems:
(1) the finite speed of light and (2) the atomic nature of matter." *

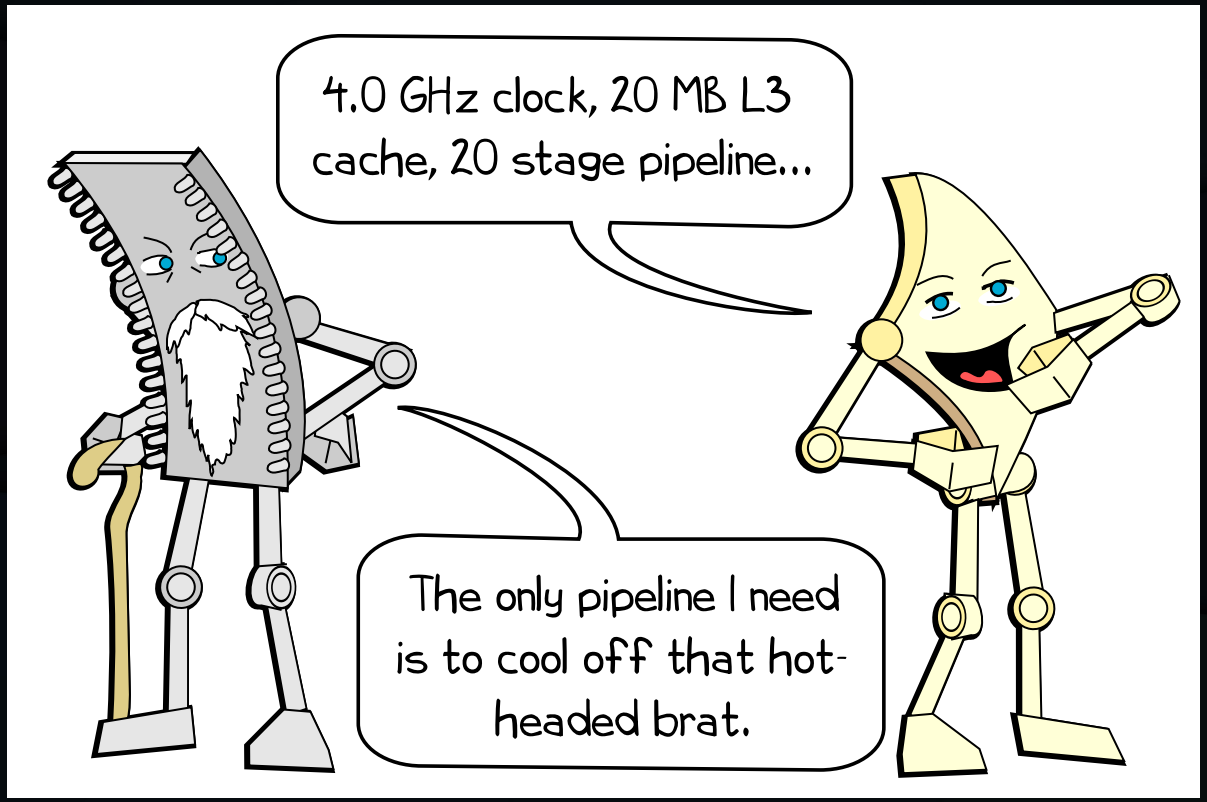
* Gordon Moore quoting Stephen Hawking

Superscalar Execution For The Win!!!

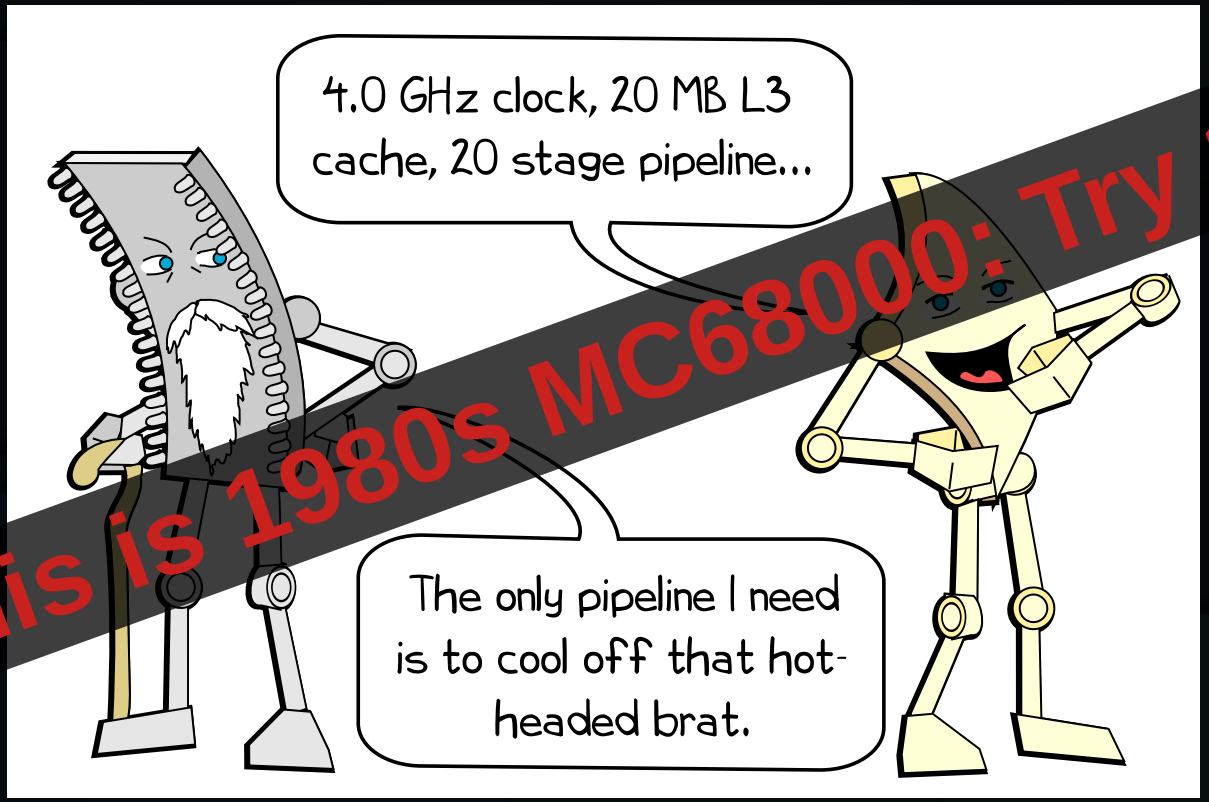


Intel Core 2 Architecture (Wikipedia user “I, Appaloosa” CC BY-SA 3.0, reformatted)

Don’t Make ‘em Like They Used To



Don’t Make ‘em Like They Used To



Paul’s and Bryan’s 1977 Schoolwork



DEC PDP-12

625 KHz CPU (1.6µs/ref)
One register (why more?)
12-bit instructions 1.6µs
4Kx12 bits core memory
No stack & no caches
512x512 graphics (green!)
No graphics frame buffer
Size of a refrigerator
sin() & cos() < 21µs

Photo in museum

“And we were #$(*&@# happy to have 4Kx12 bits!!!”

https://www.rcsri.org/collection/pdp-12/skylark-ps.txt
Photo: The Retro-Computing Society of Rhode Island, Inc.

Paul’s and Bryan’s 1977 Schoolwork



DEC PDP-12

625 KHz CPU (1.6µs/ref)
One register (why more?)
12-bit instructions 1.6µs
4Kx12bits core memory
No stack & no caches
512x512 graphics (green!)
No graphics frame buffer
Size of a refrigerator
sin() & cos() < 21µs

Photo in museum

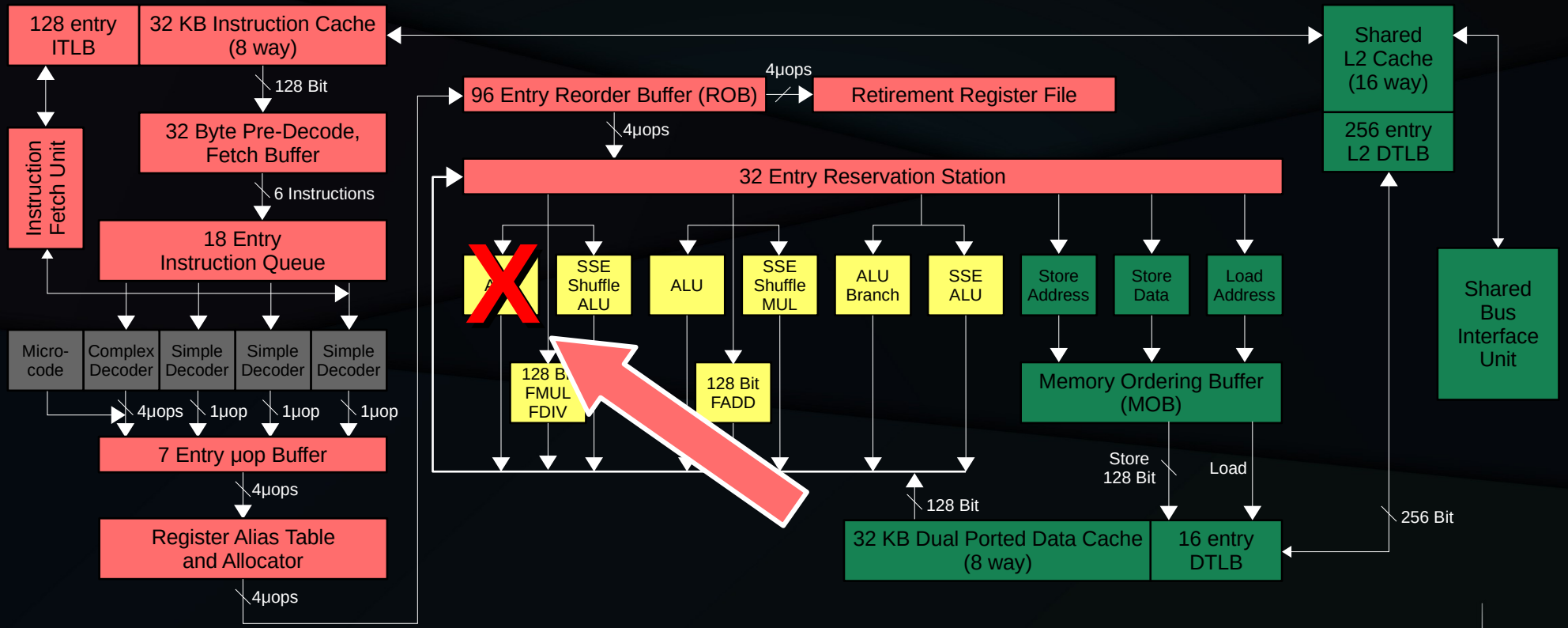
“And we were #$(*&@# happy to have 4Kx12 bits!!!”

https://www.rcsri.org/collection/pdp-12/skylark-ps.txt
Photo: The Retro-Computing Society of Rhode Island, Inc.

Account For All CPU Complexity???

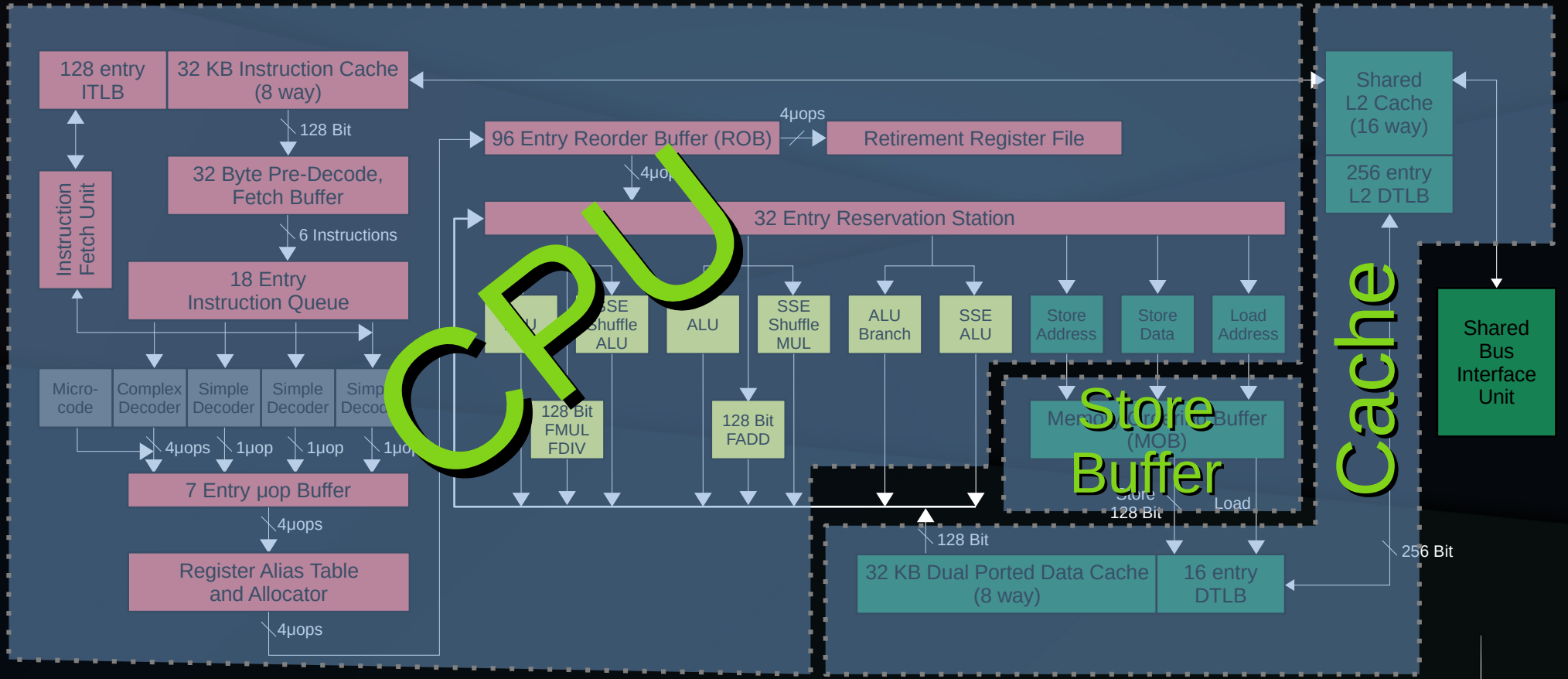
- Sometimes, yes! (Assembly language!)
- But we also need portability: CPUs change
  - From family to family
  - With each revision of silicon
  - To work around hardware bugs
  - As a given physical CPU ages

One of the ALUs Might Be Disabled



Intel Core 2 Architecture (Wikipedia user “I, Appaloosa” CC BY-SA 3.0, reformatted)

Thus, Simple Portable CPU Model



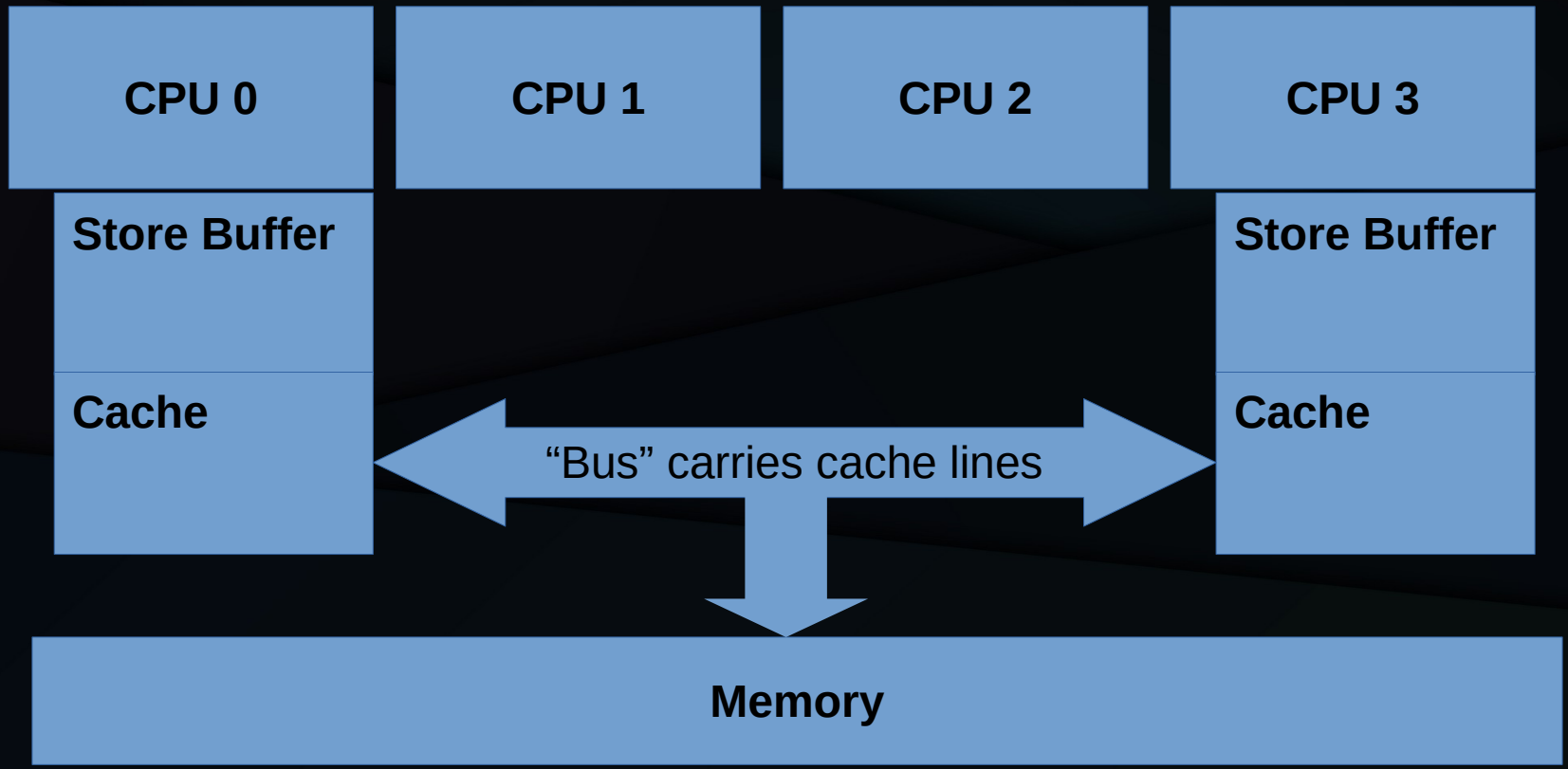
Intel Core 2 Architecture (Wikipedia user “I, Appaloosa” CC BY-SA 3.0, reformatted and remixed)

CPU, Store Buffer, Cache, and “Bus”

CPU, Store Buffer, Cache, and “Bus”

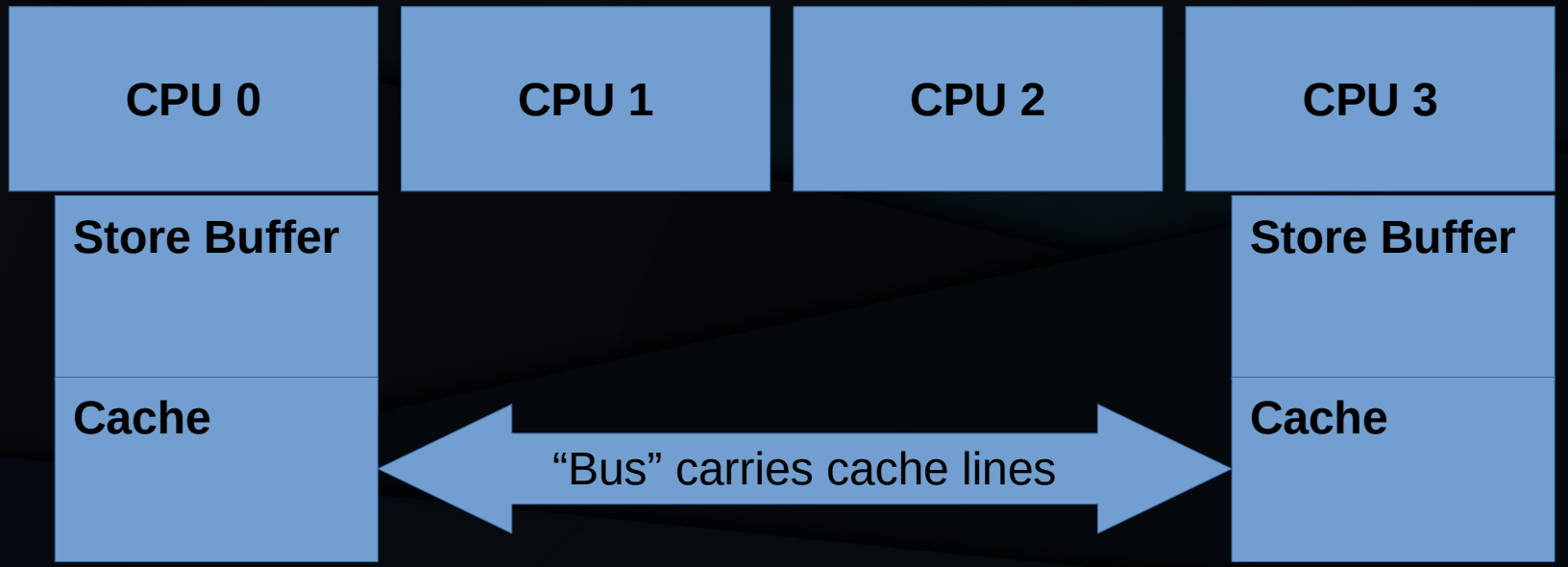
- CPU computes with words (e.g., 64 bits)
- Store buffer holds words waiting for cache lines
- Cache is a hardware hash table containing “cache lines” (e.g., 64 bytes which is 512 bits)
- “Bus” (AKA “interconnect”) communicates cache lines among CPUs’ caches and memory

CPU, Store Buffer, Cache, and “Bus”



Communication is slow! So use locality of reference to avoid unnecessary communication!!!

CPU, Store Buffer, Cache, and “Bus”



In later examples, data will have no chance to reach memory

Communication is slow! So use locality of reference to avoid unnecessary communication!!!

Let’s Do Some Concurrent Counting!!!

Just Count Concurrently!!!

```
unsigned long ctr;

void inc_count(void)
{
    ctr++;
}

void read_count(void)
{
    return ctr;
}
```

Just Count Concurrently!!!

```
unsigned long ctr;

void inc_count(void)
{
    ctr++;
}

void read_count(void)
{
    return ctr;
}
```

Anyone care to critique this code?

Just Count Concurrently!!!

```
unsigned long ctr;

void inc_count(void)
{
    ctr++;
}

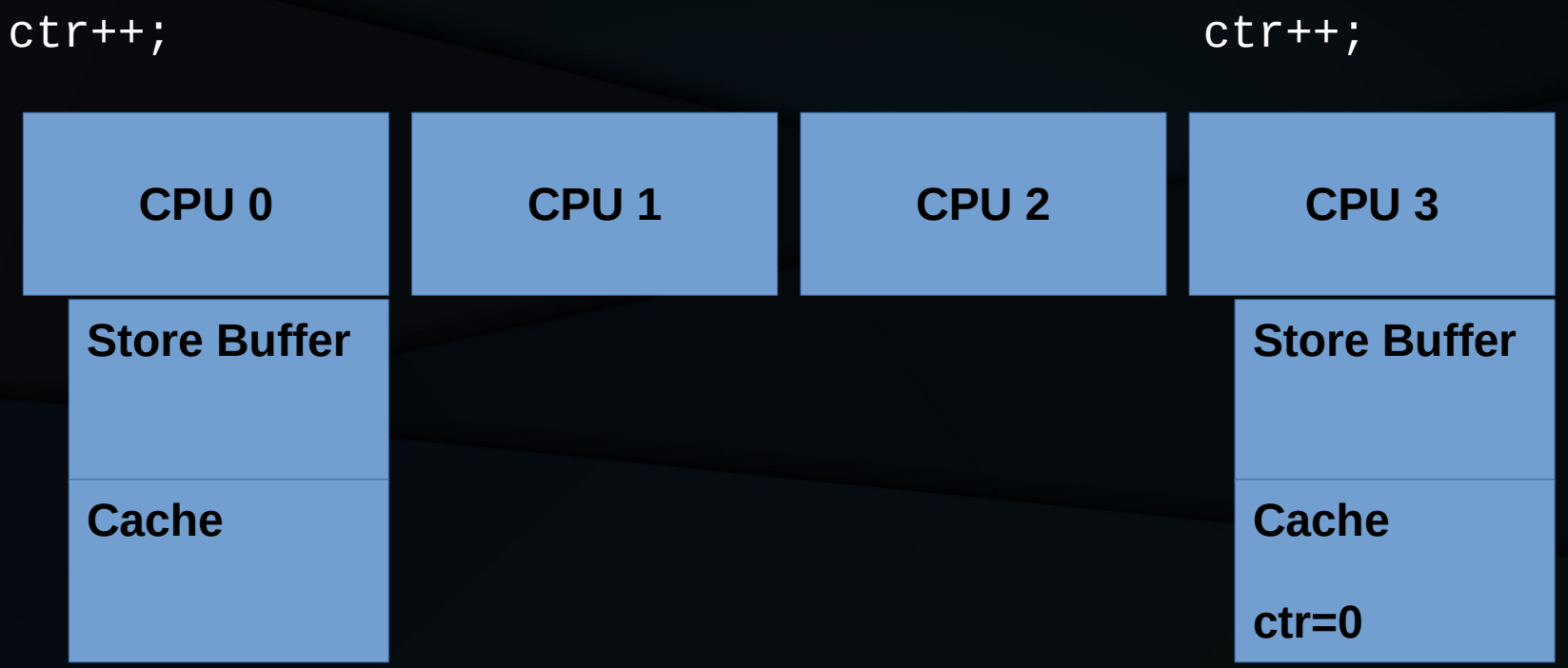
void read_count(void)
{
    return ctr;
}
```

Running on 6 CPUs loses 87% of the counts, which is not good!

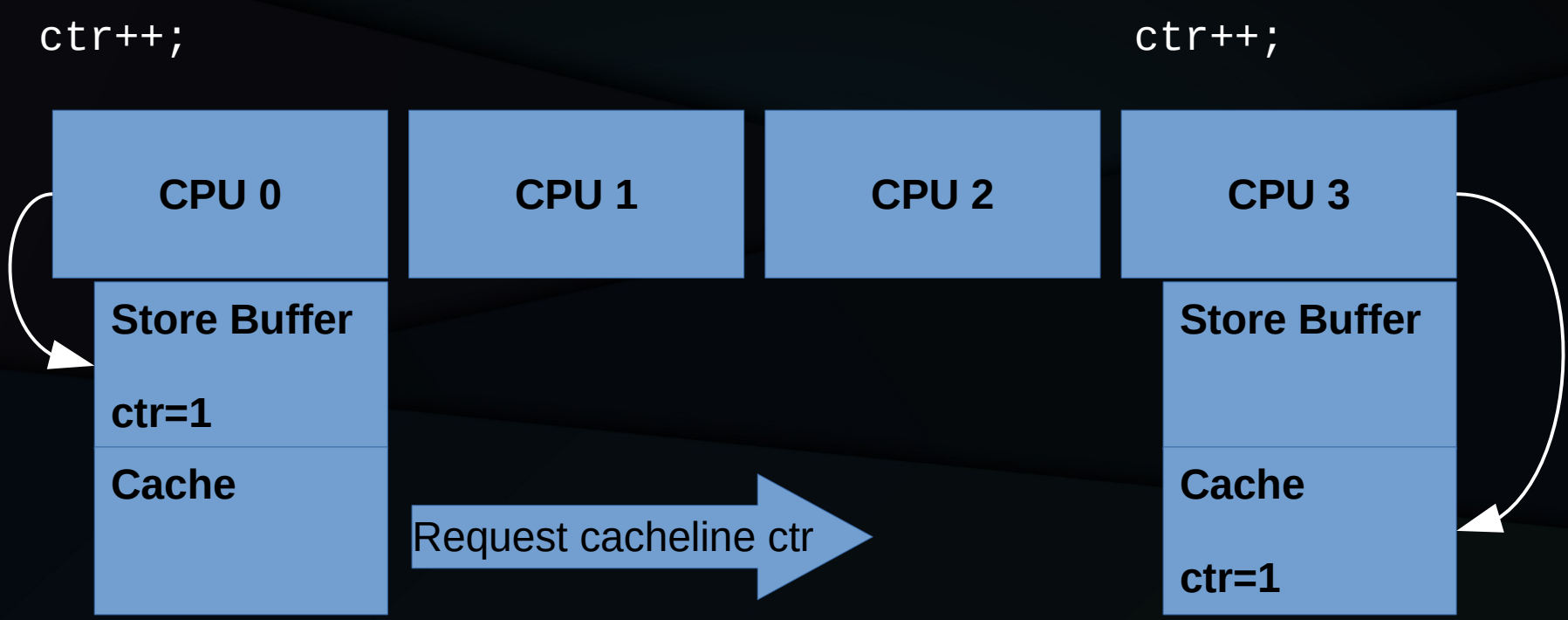
Anyone care to critique this code?

Why Are Counts Lost???

Why Are Counts Lost??? (1/7)

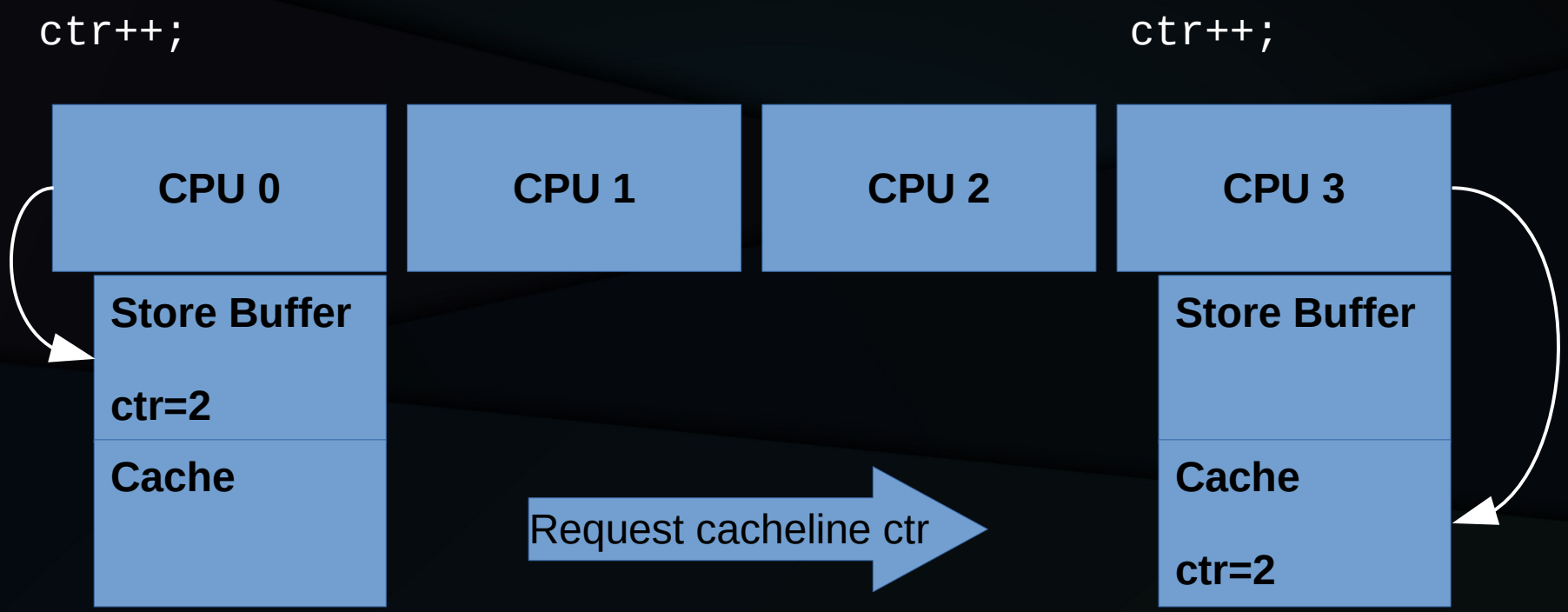


Why Are Counts Lost??? (2/7)

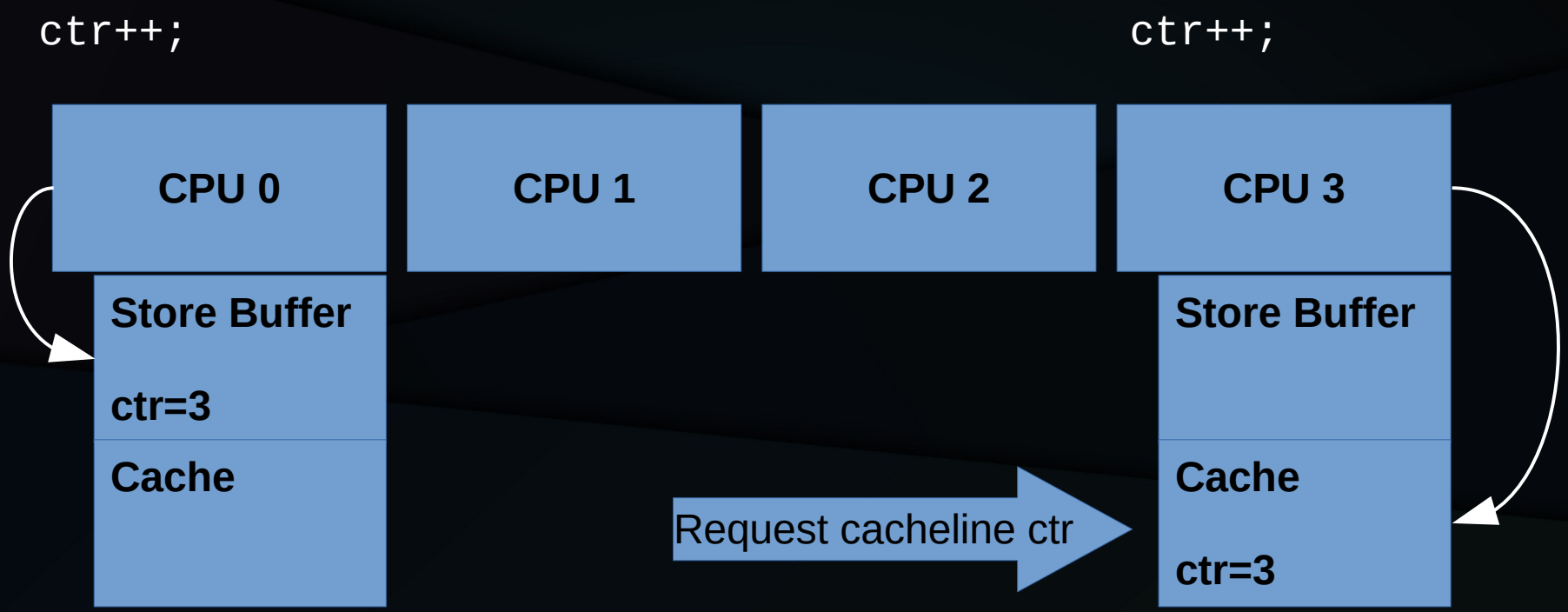


The store buffer allows increments to completes quickly!!! Take that, laws of physics!!!

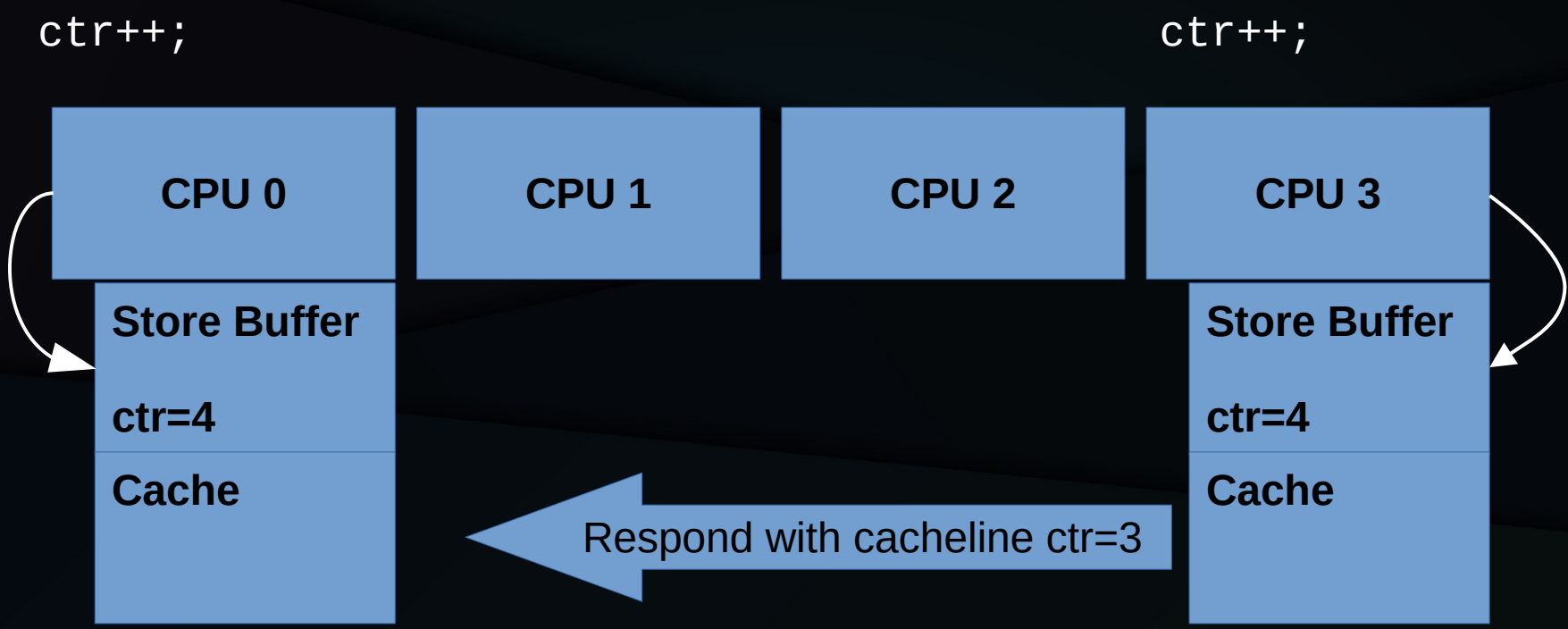
Why Are Counts Lost??? (3/7)



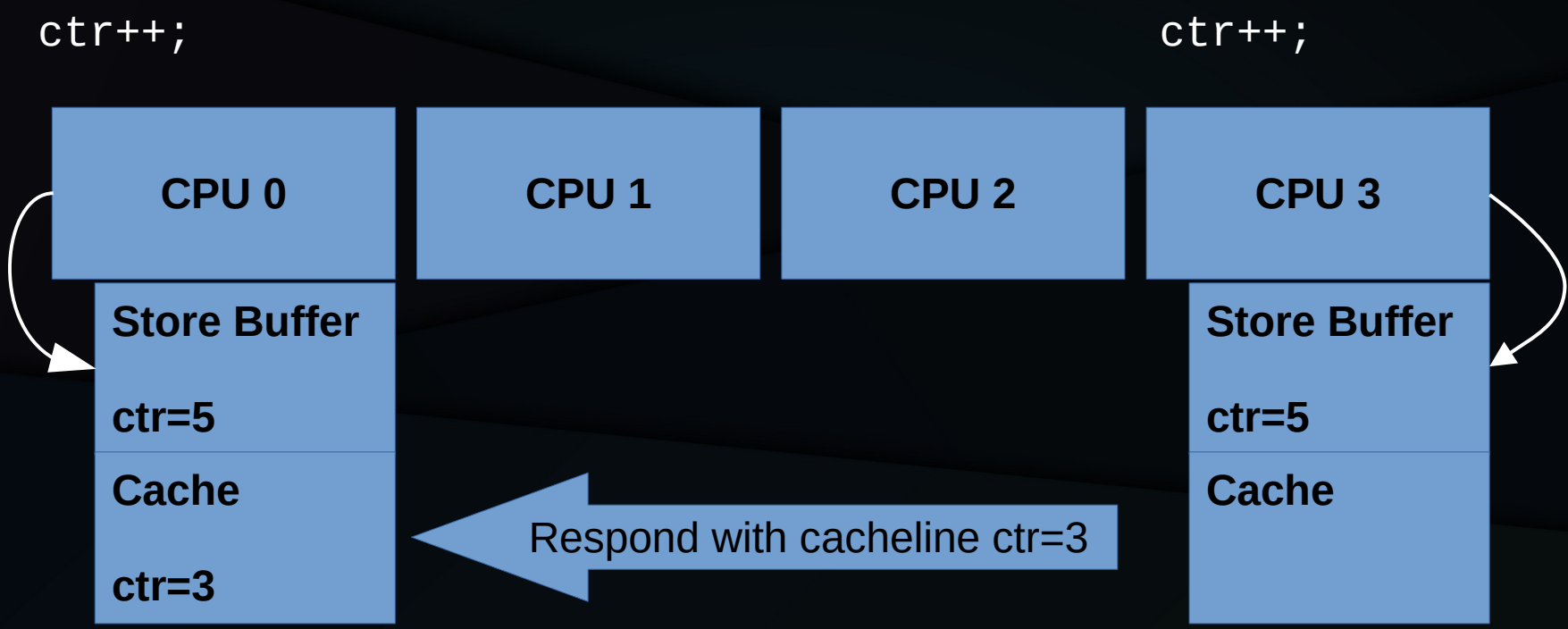
Why Are Counts Lost??? (4/7)



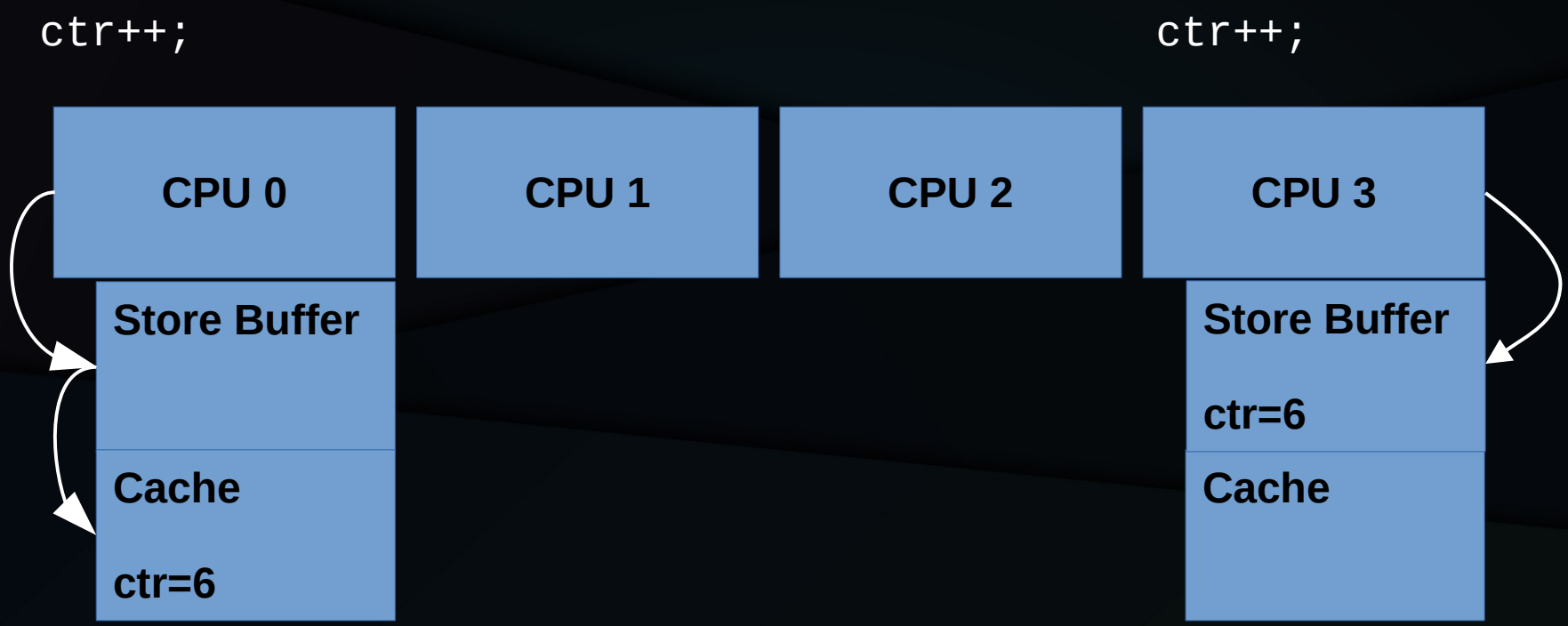
Why Are Counts Lost??? (5/7)



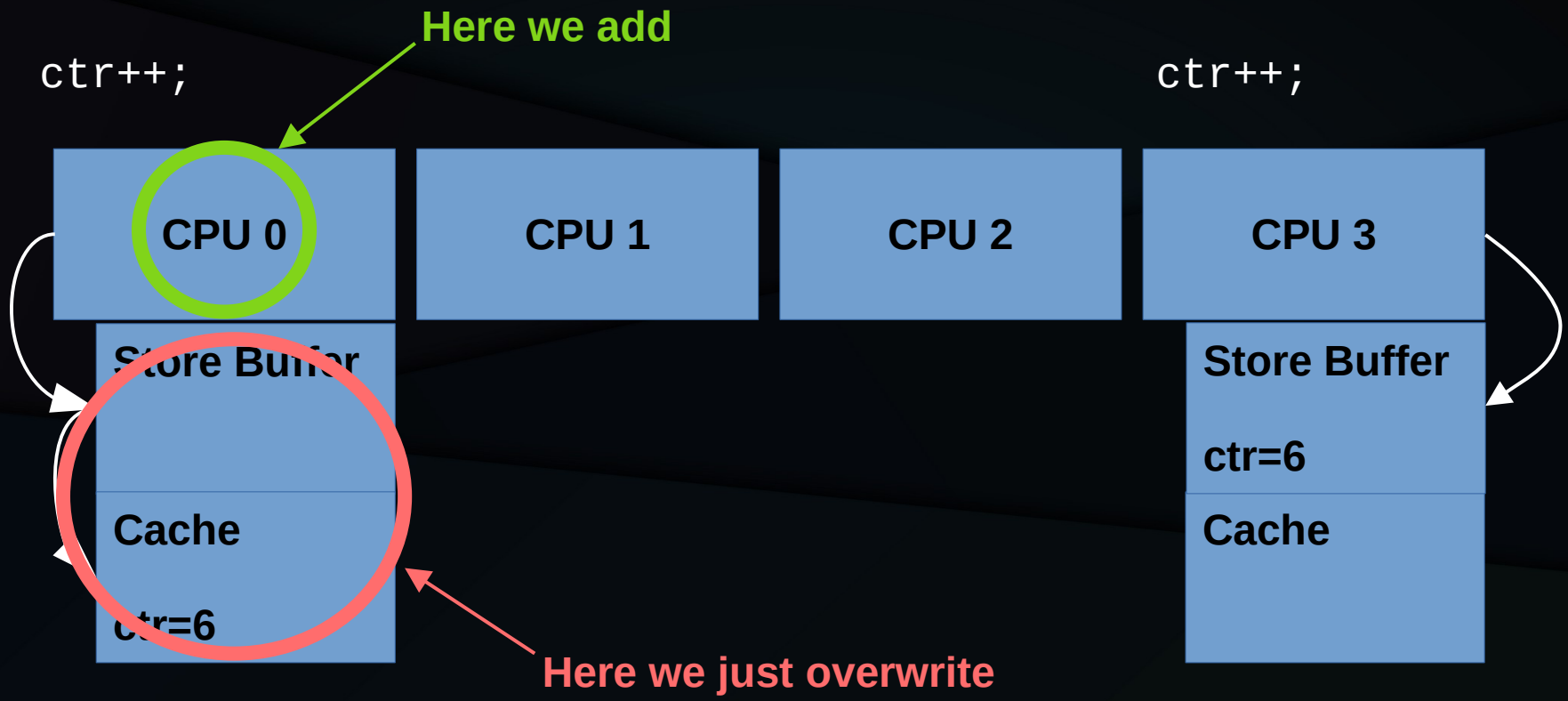
Why Are Counts Lost??? (6/7)



Why Are Counts Lost??? (7/7)



Why Are Counts Lost??? (7/7 Redux)



Quick write completion, sort of. Laws of physics: Slow or lost counts!!!

Why Are Counts Lost??? (7/7 Redux)



Quick write completion, sort of. Laws of physics: Slow or lost counts!!!

Just Count Atomically!!!

Just Count Atomically!!!

```
atomic_t ctr;

void inc_count(void)
{
    atomic_inc(&ctr);
}

void read_count(void)
{
    return atomic_read(&ctr);
}
```

Just Count Atomically!!!

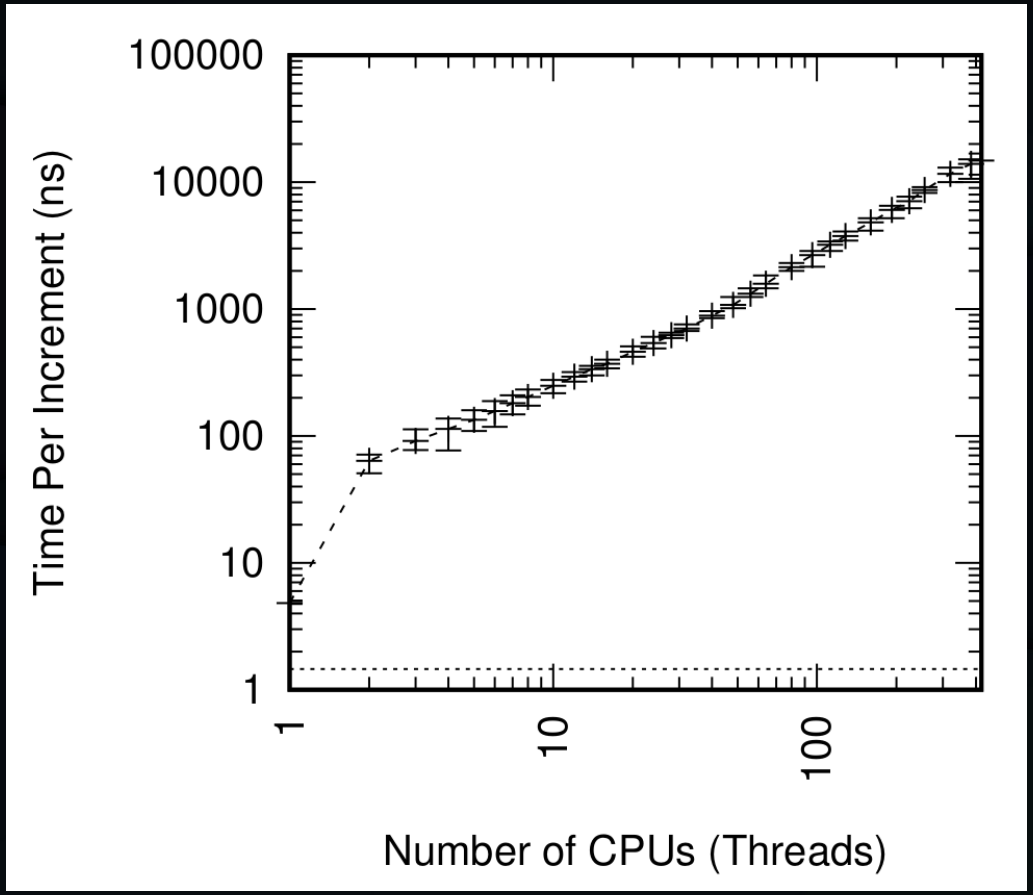
```
atomic_t ctr;

void inc_count(void)
{
    atomic_inc(&ctr);
}

void read_count(void)
{
    return atomic_read(&ctr);
}
```

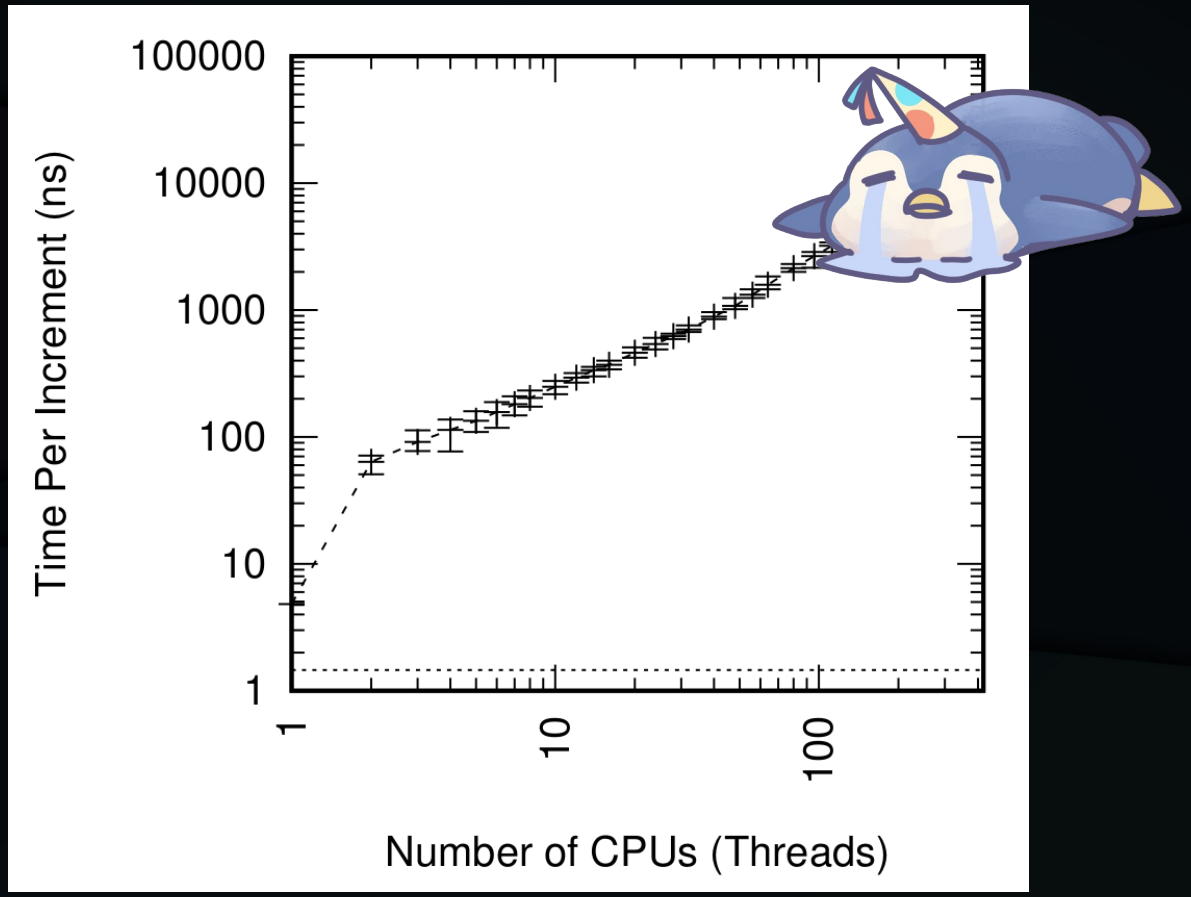
Anyone care to critique this code?

Counting Atomically Does Not Scale



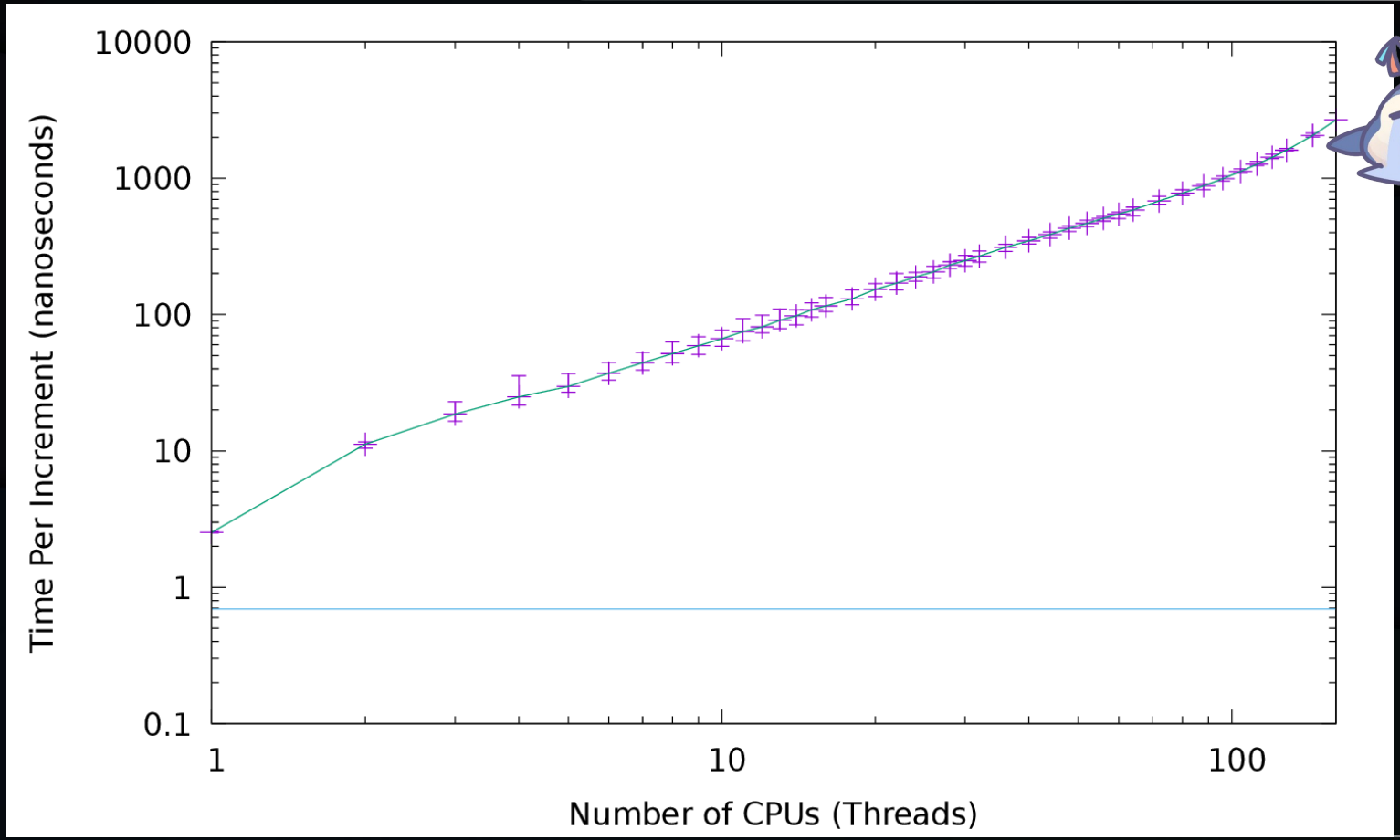
448-CPU Intel(R) Xeon(R) Platinum 8176 @ 2.10GHz

Counting Atomically Does Not Scale



448-CPU Intel(R) Xeon(R) Platinum 8176 @ 2.10GHz

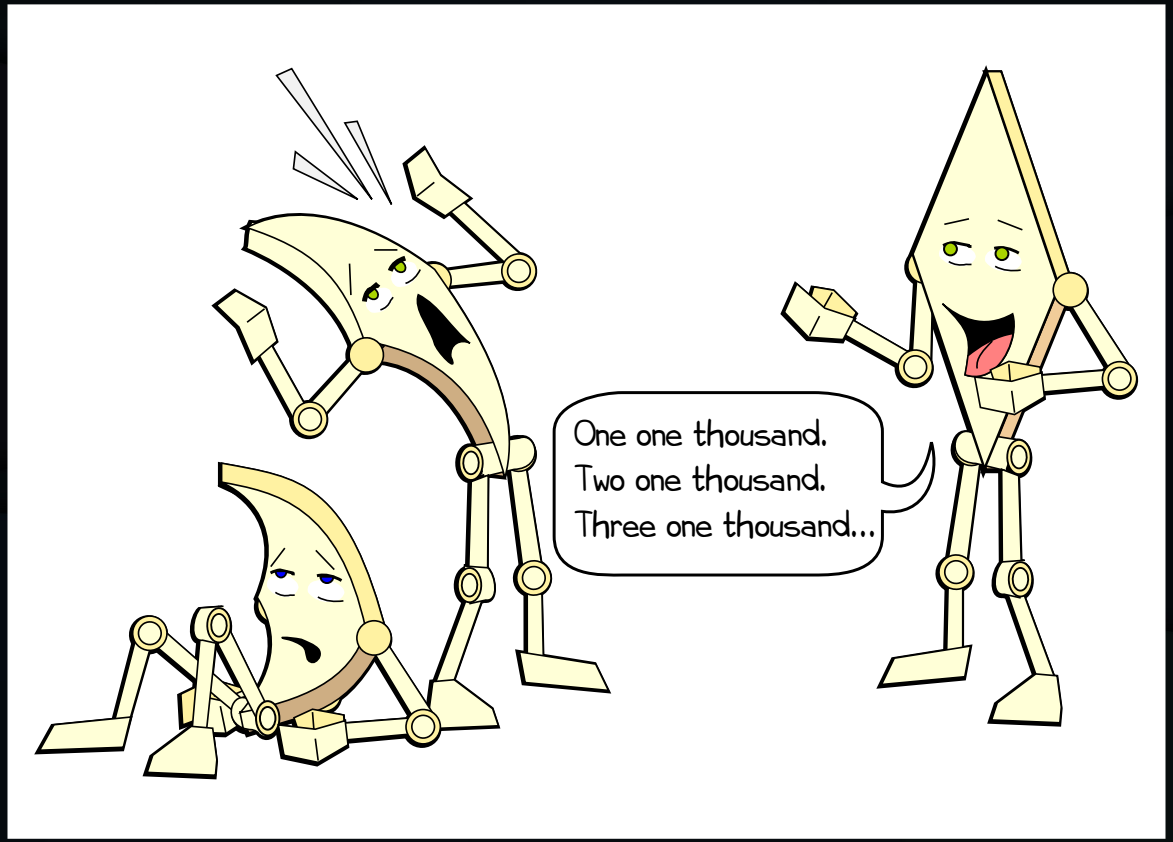
And *Still* Does Not Scale!!!



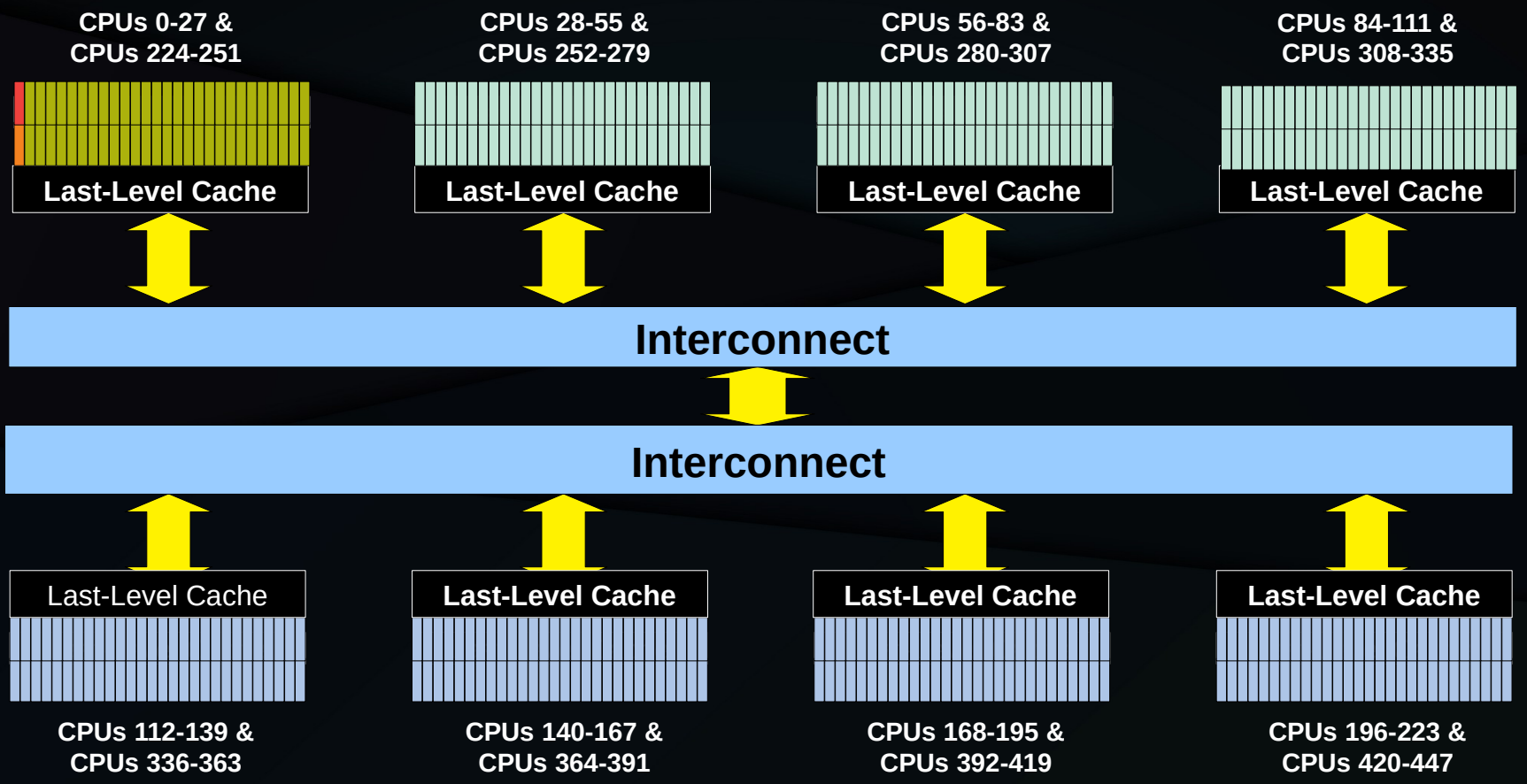


166-CPU AMD EPYC-Milan Processor @ 2.0GHz

Spare a Thought for Those CPUs!

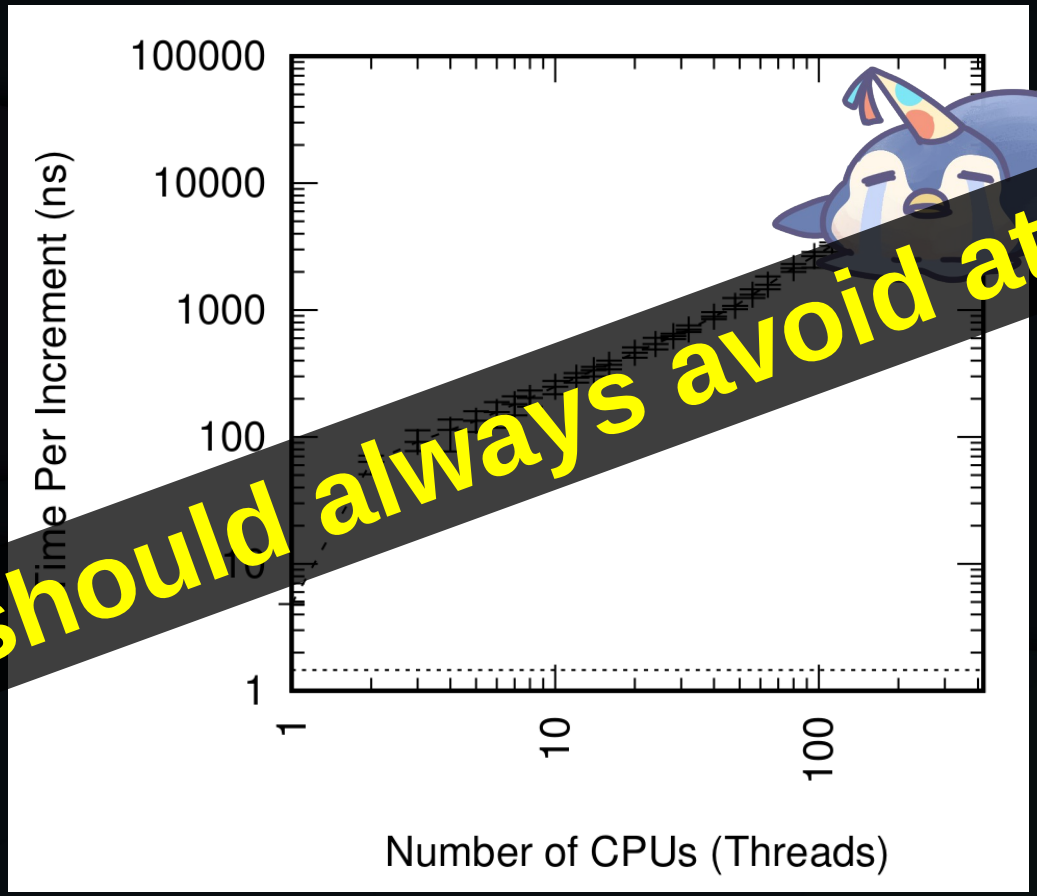


Cache Line Thrashes Among CPUs!!!



HW optimizations? https://kernel.org/pub/linux/kernel/people/paulmck/perfbook/perfbook-e2.pdf Chapter 5 Quick Quiz 5.11

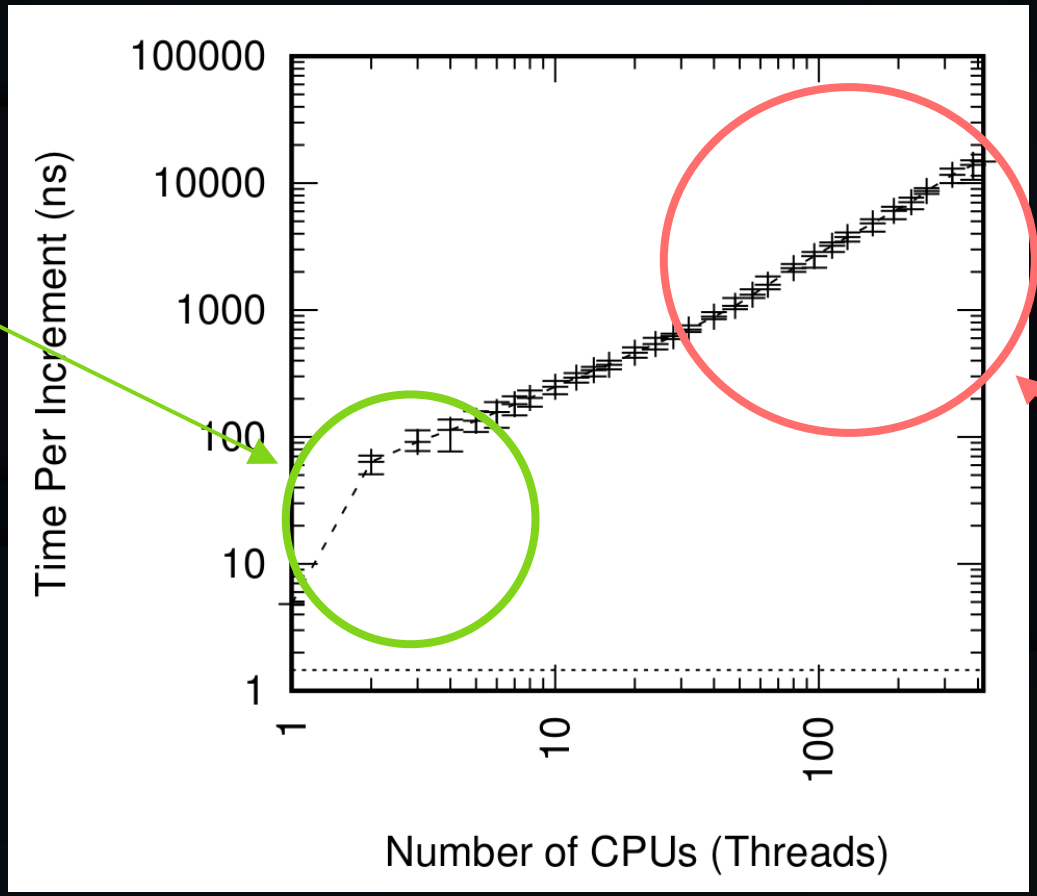
Counting Atomically Does Not Scale



448-CPU Intel(R) Xeon(R) Platinum 8176 CPU @ 2.10GHz

Counting Atomically ***Can*** Be OK...

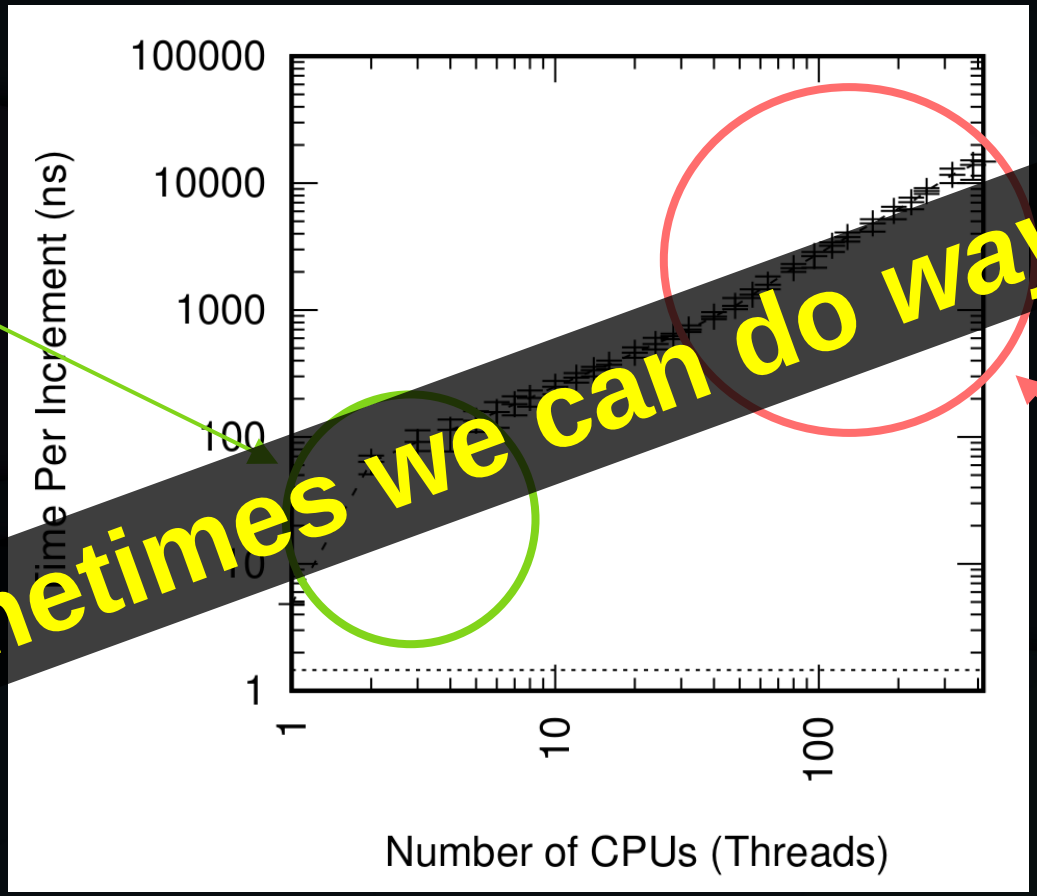
Here is just fine!



Here, not so much!!!

448-CPU Intel(R) Xeon(R) Platinum 8176 CPU @ 2.10GHz

Counting Atomically ***Can*** Be OK...



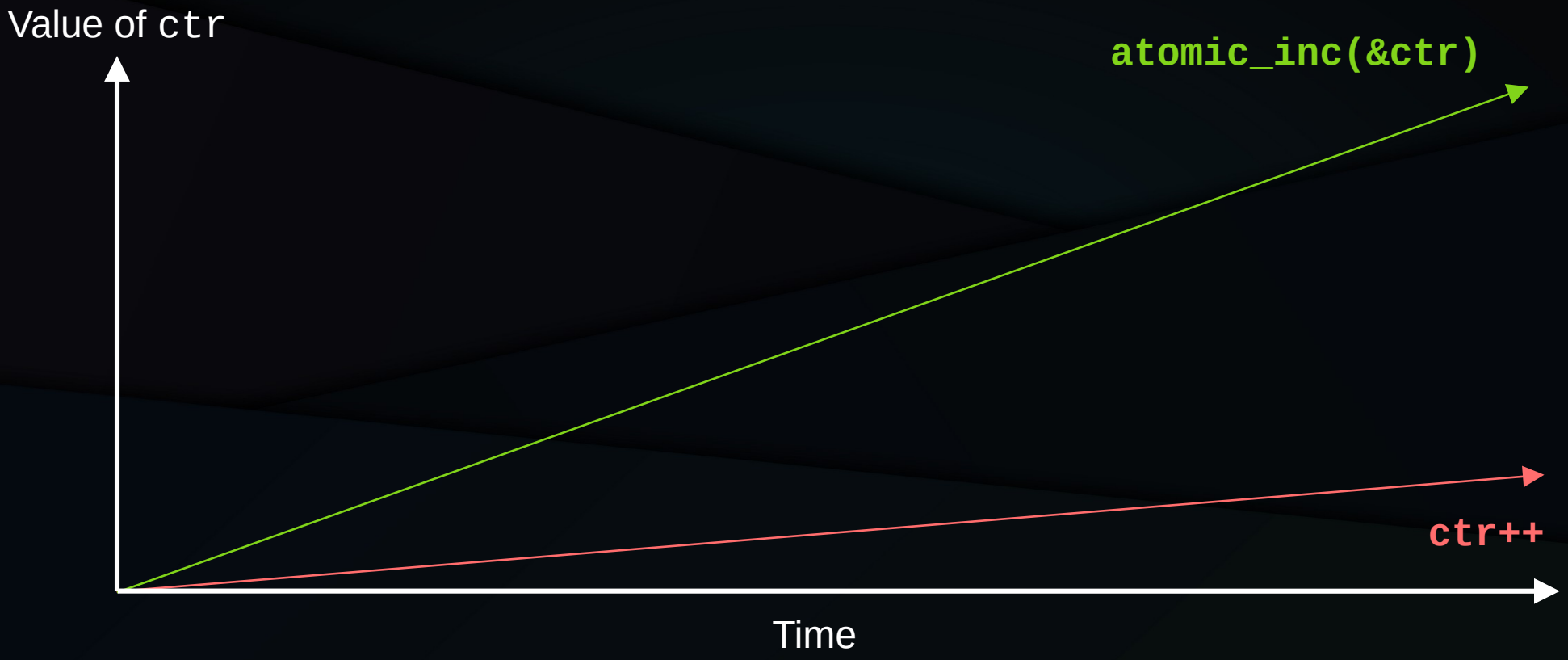
448-CPU Intel(R) Xeon(R) Platinum 8176 CPU @ 2.10GHz

How Accurate is the Reader's Value?

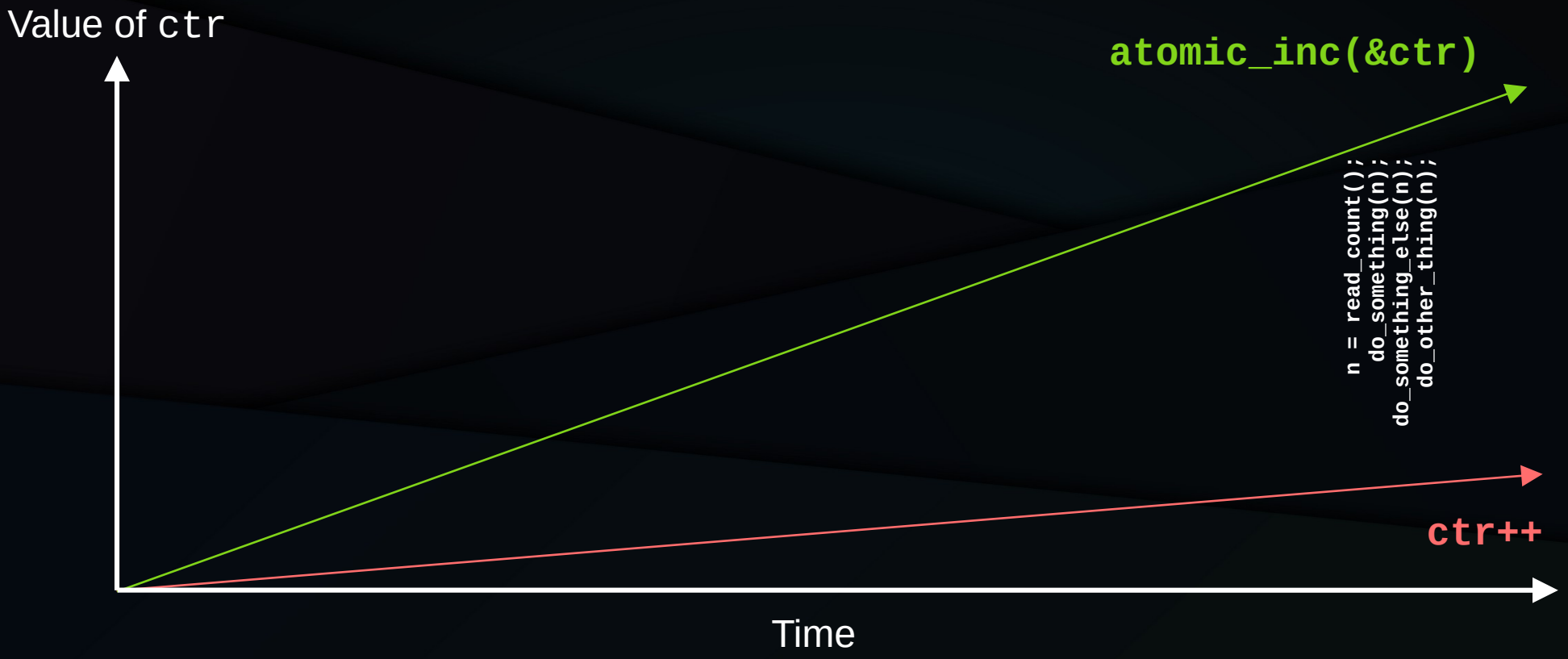
```
n = read_count();
do_something(n);       // Here?
do_something_else(n); // How about here???
do_other_thing(n);    // And here???
```

Jjj

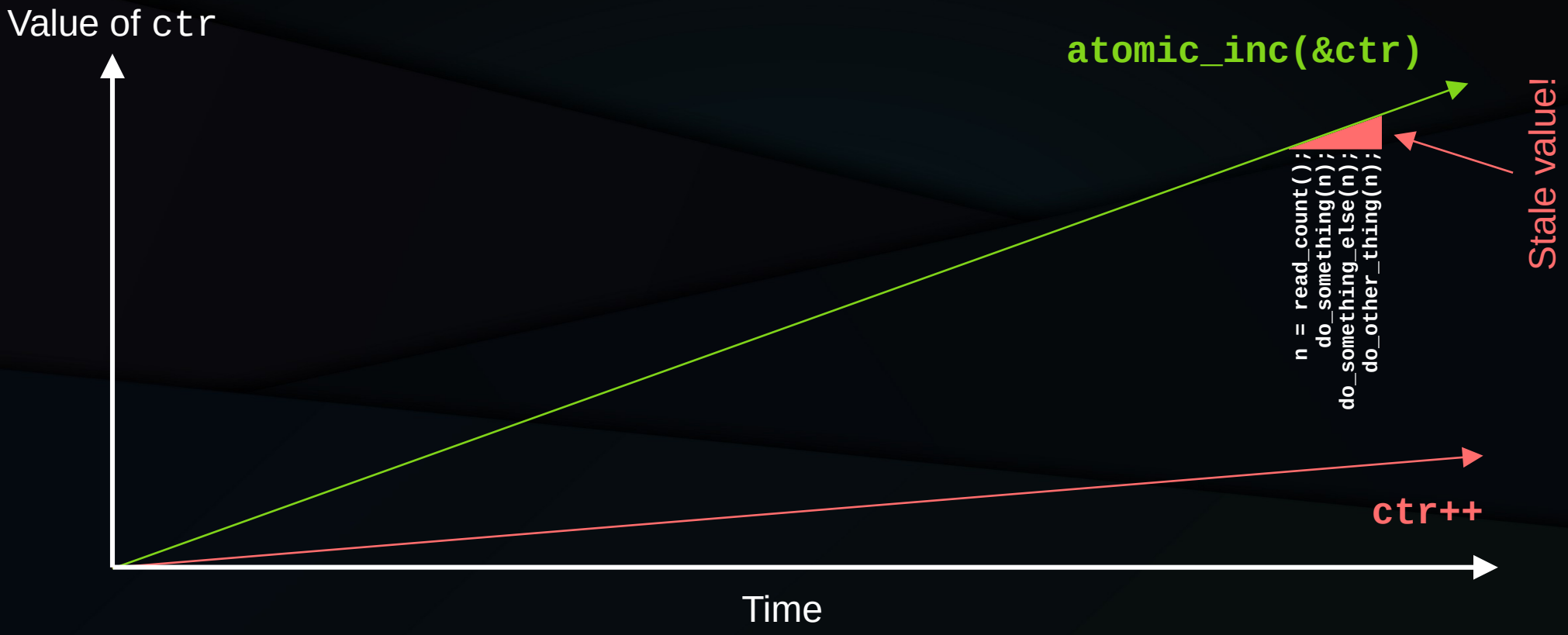
How Accurate is the Reader's Value?



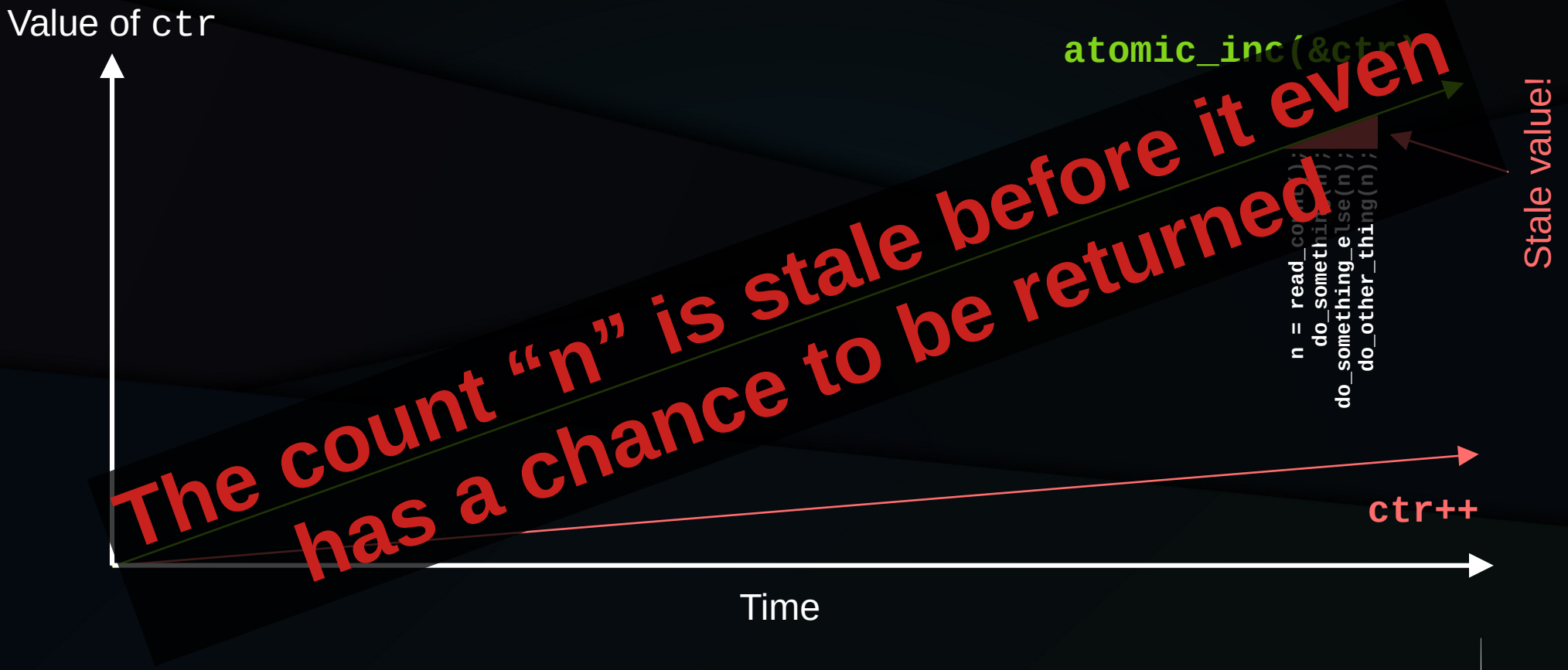
How Accurate is the Reader's Value?



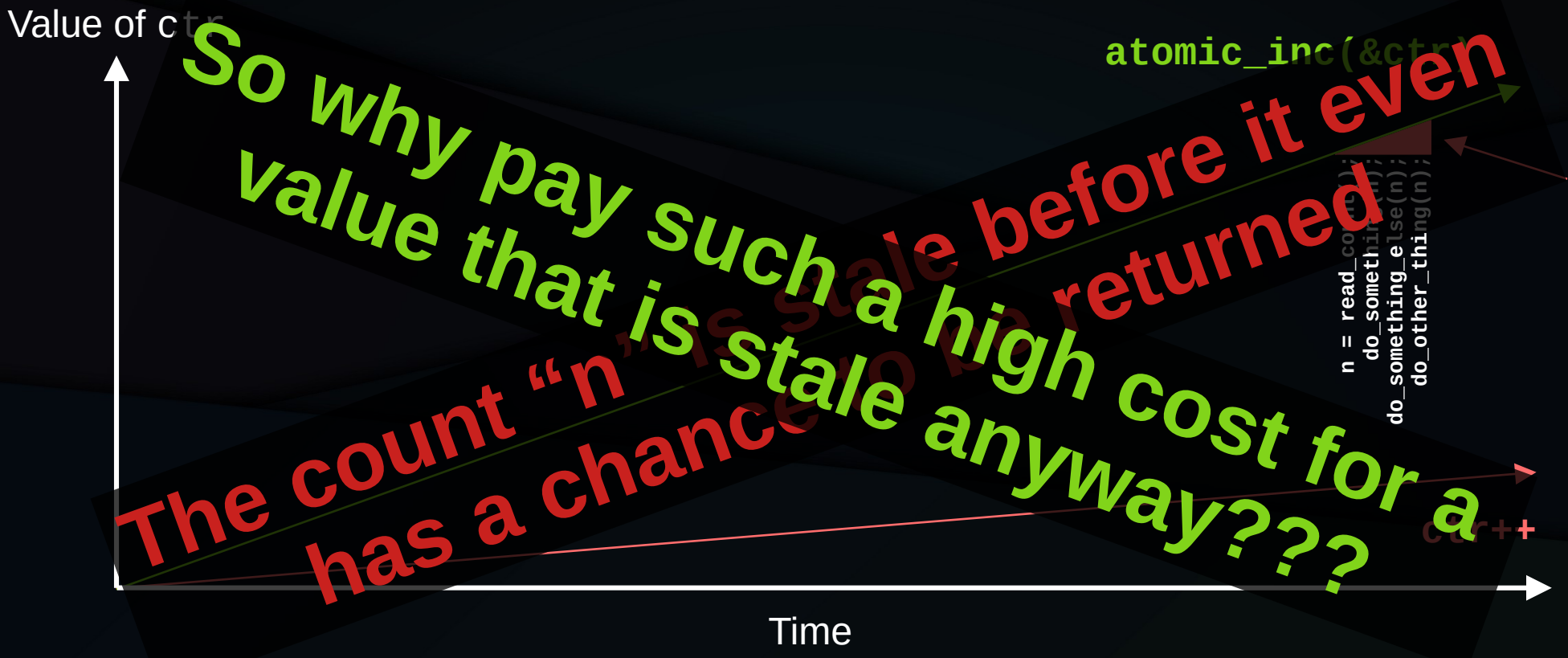
How Accurate is the Reader's Value?



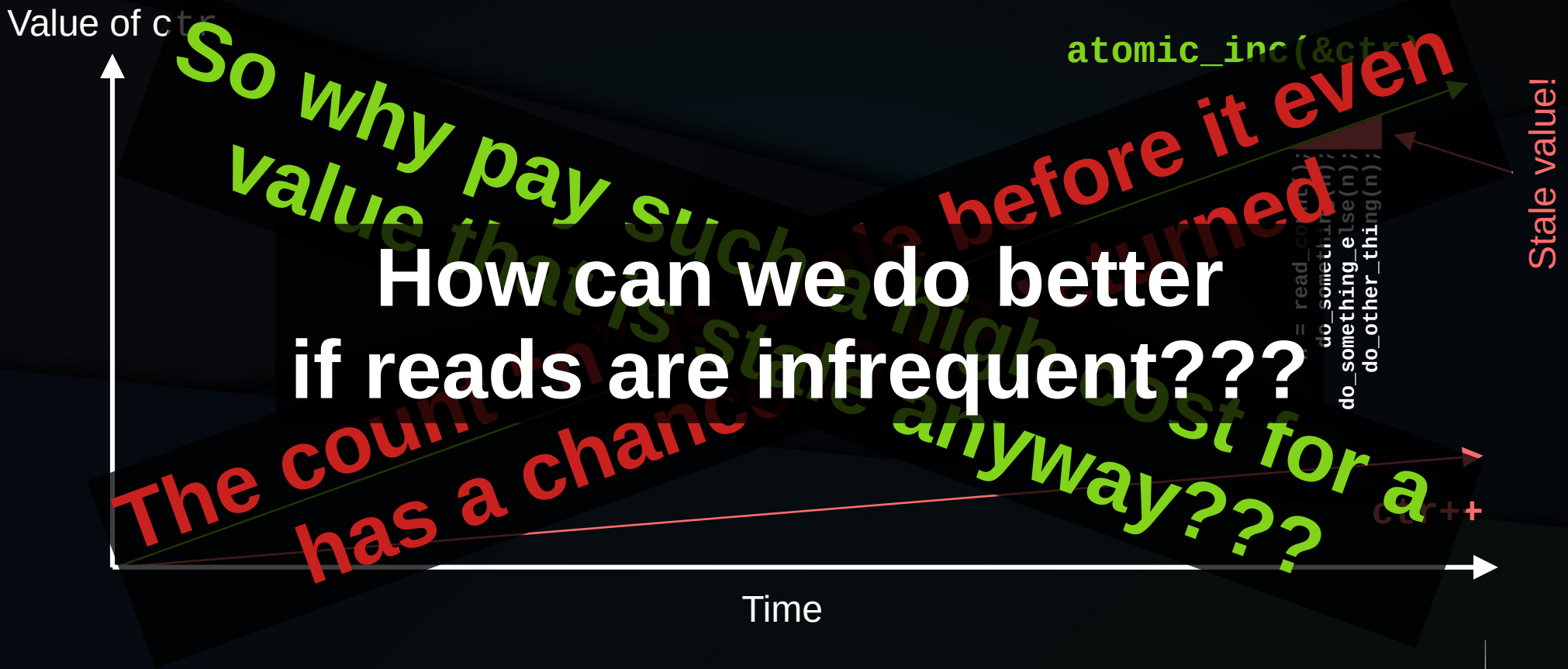
How Accurate is the Reader's Value?



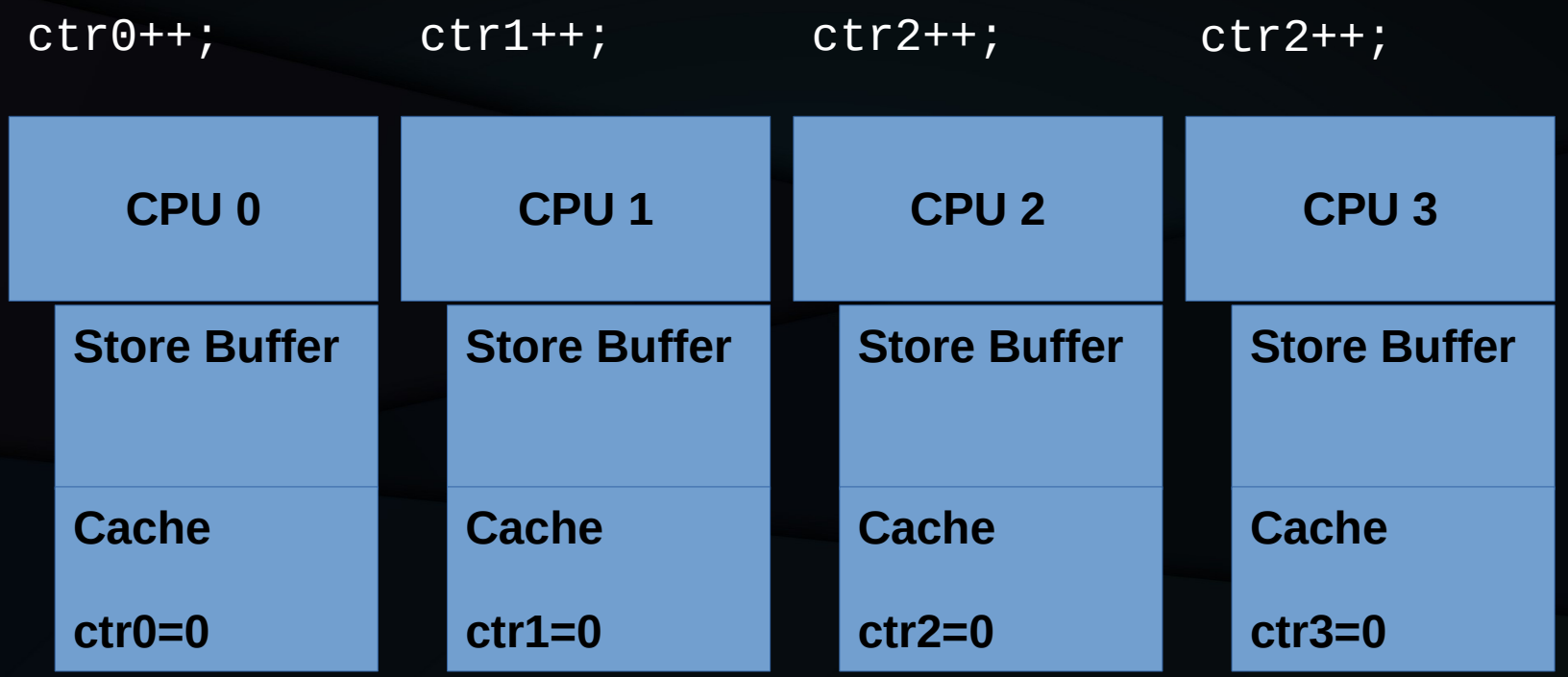
How Accurate is the Reader's Value?



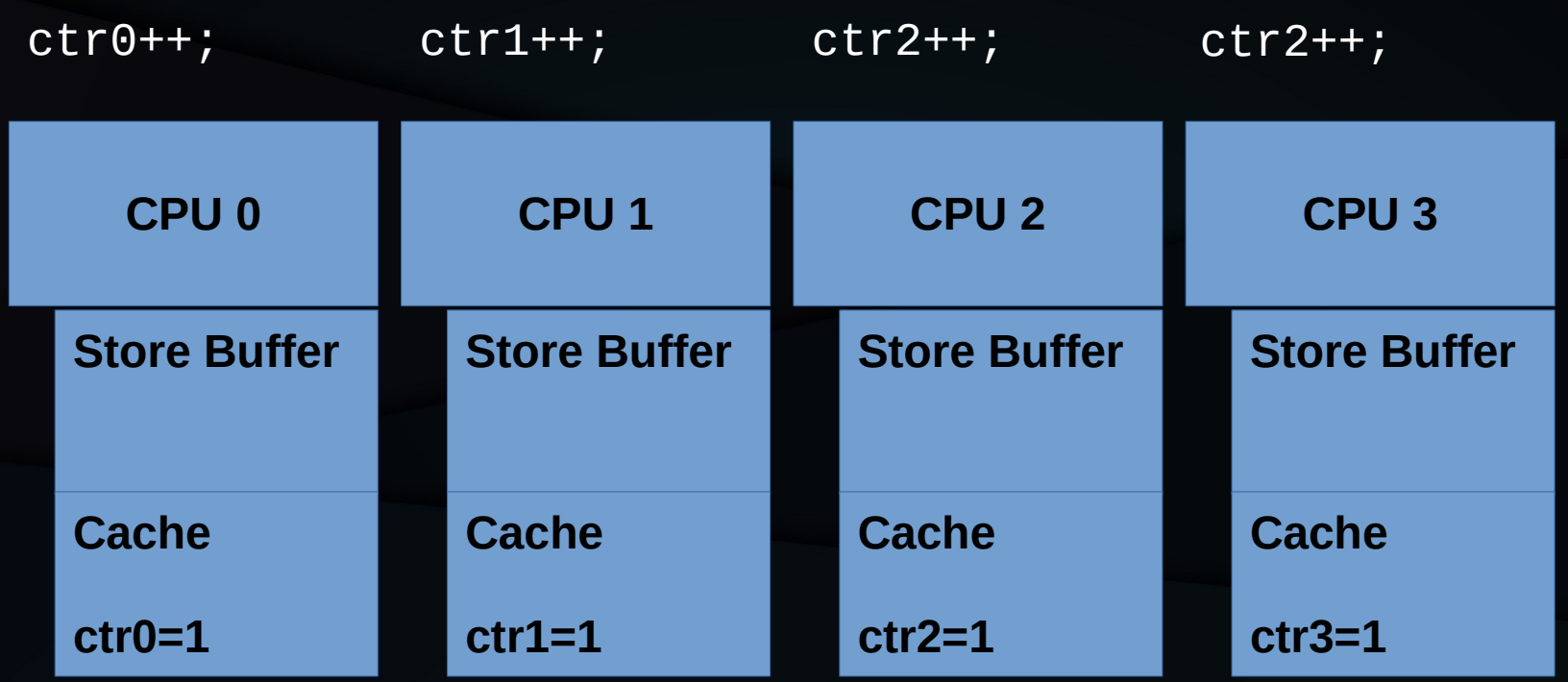
How Accurate is the Reader's Value?



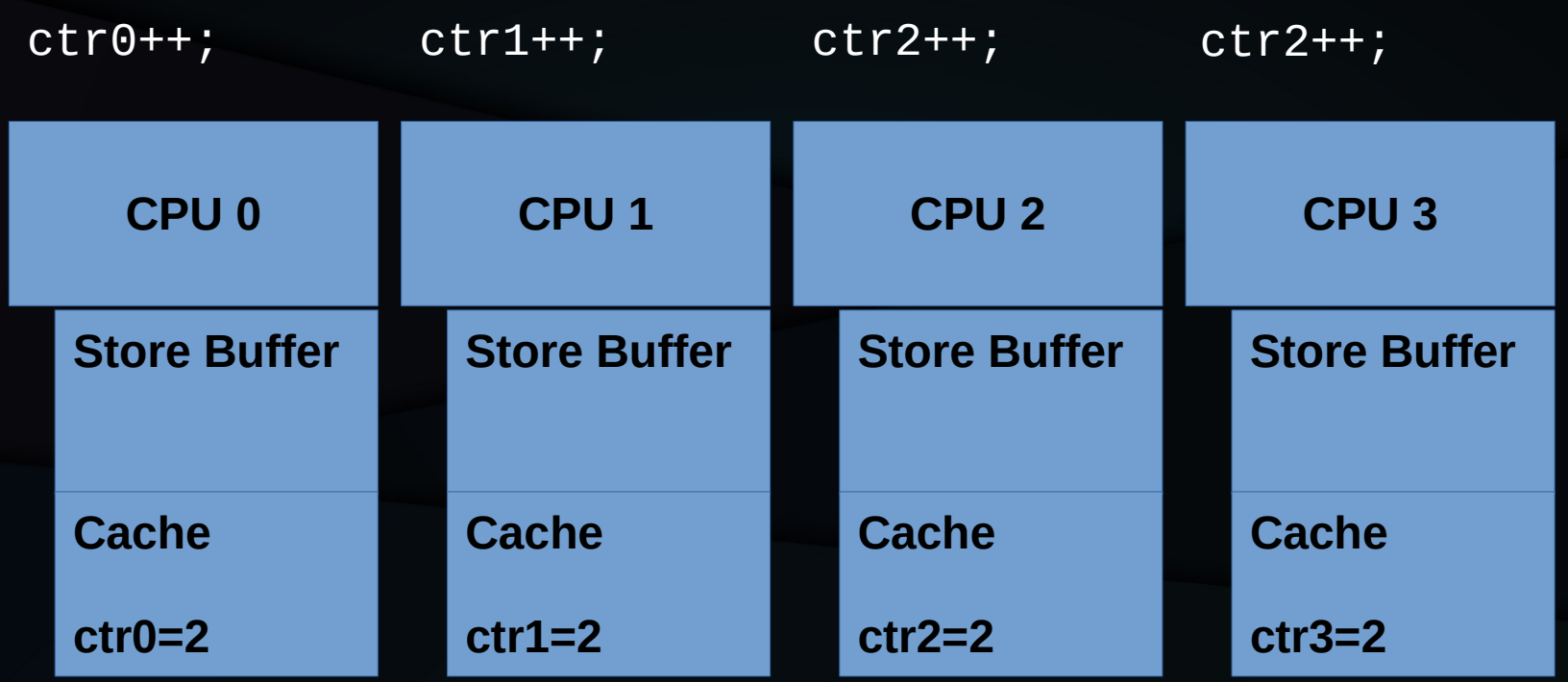
How to Count Better??? (1/7)



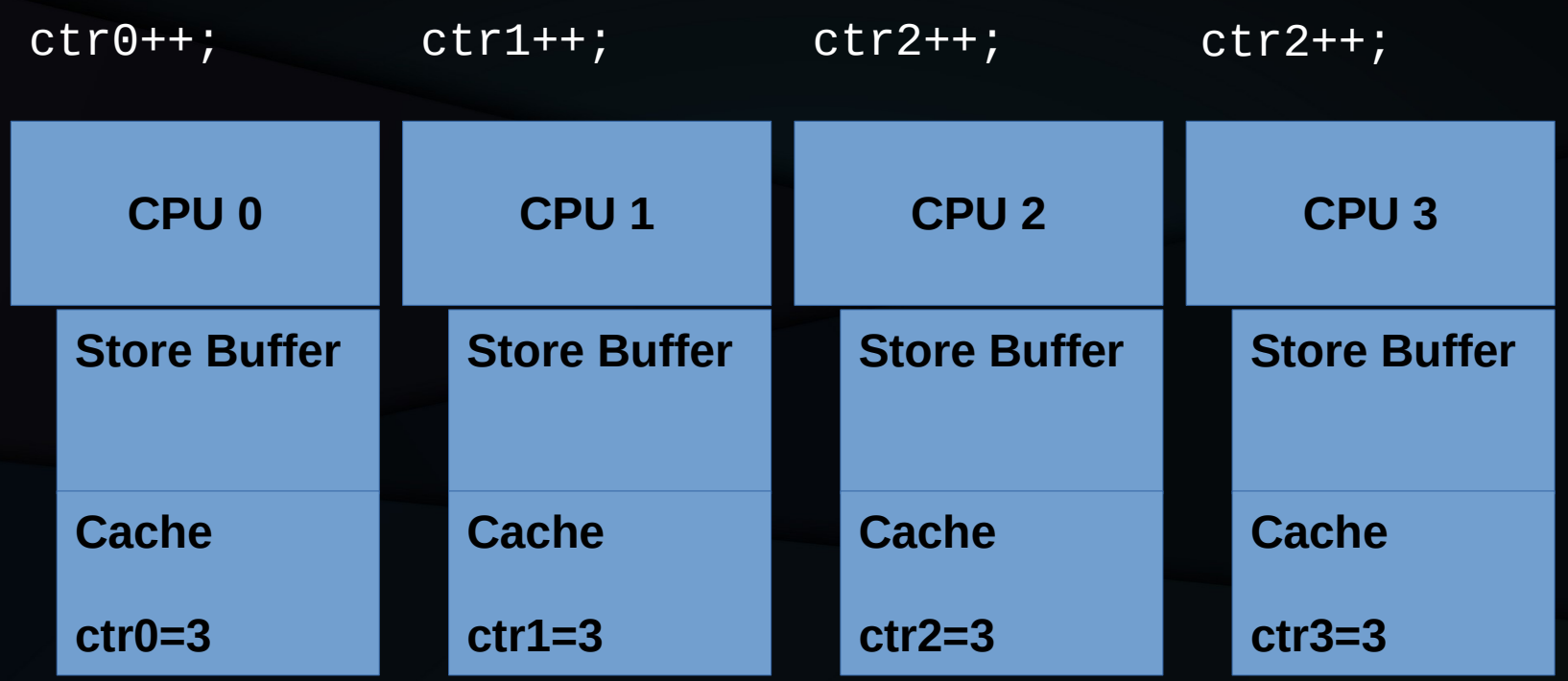
How to Count Better??? (2/7)



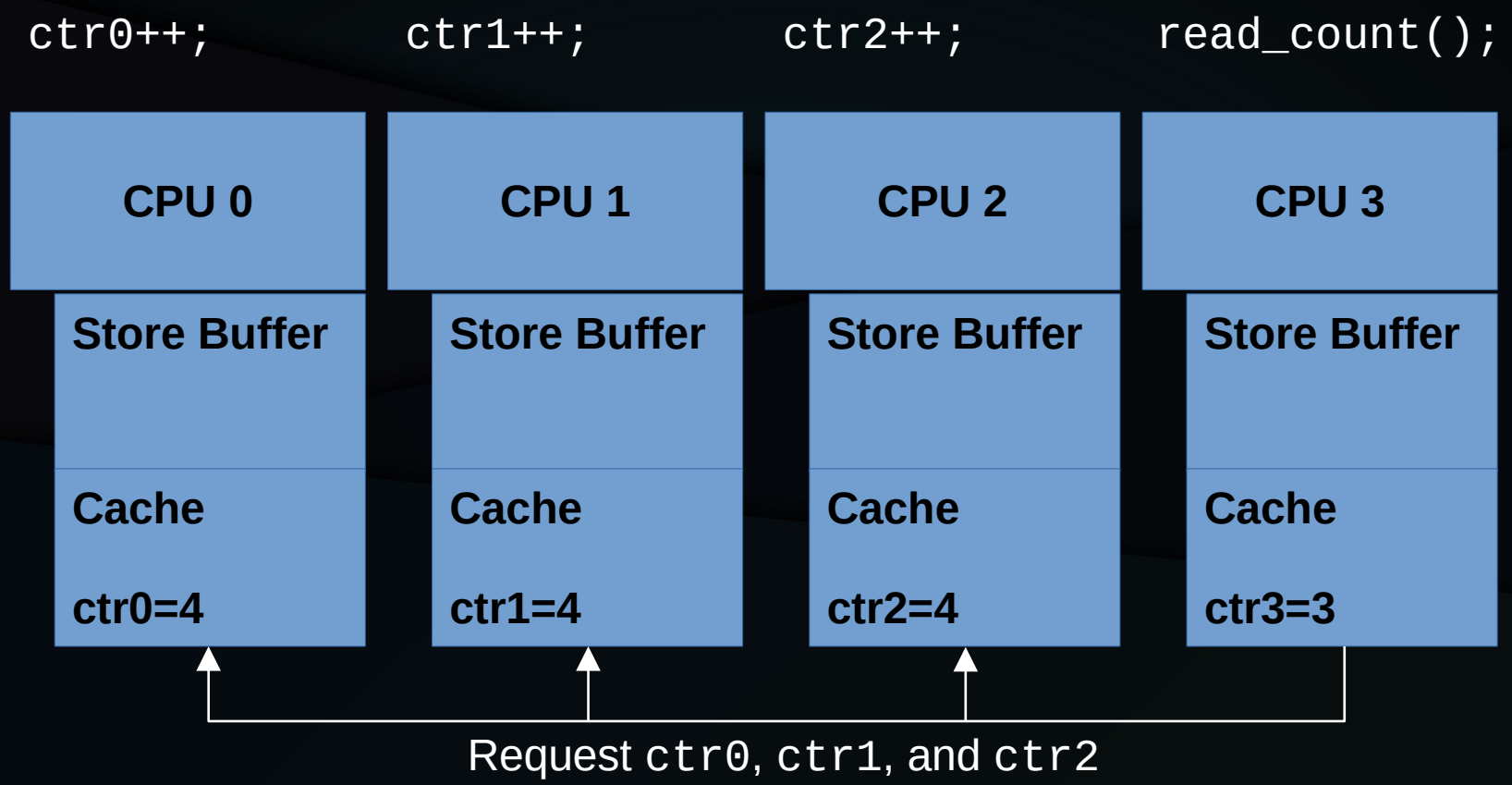
How to Count Better??? (3/7)



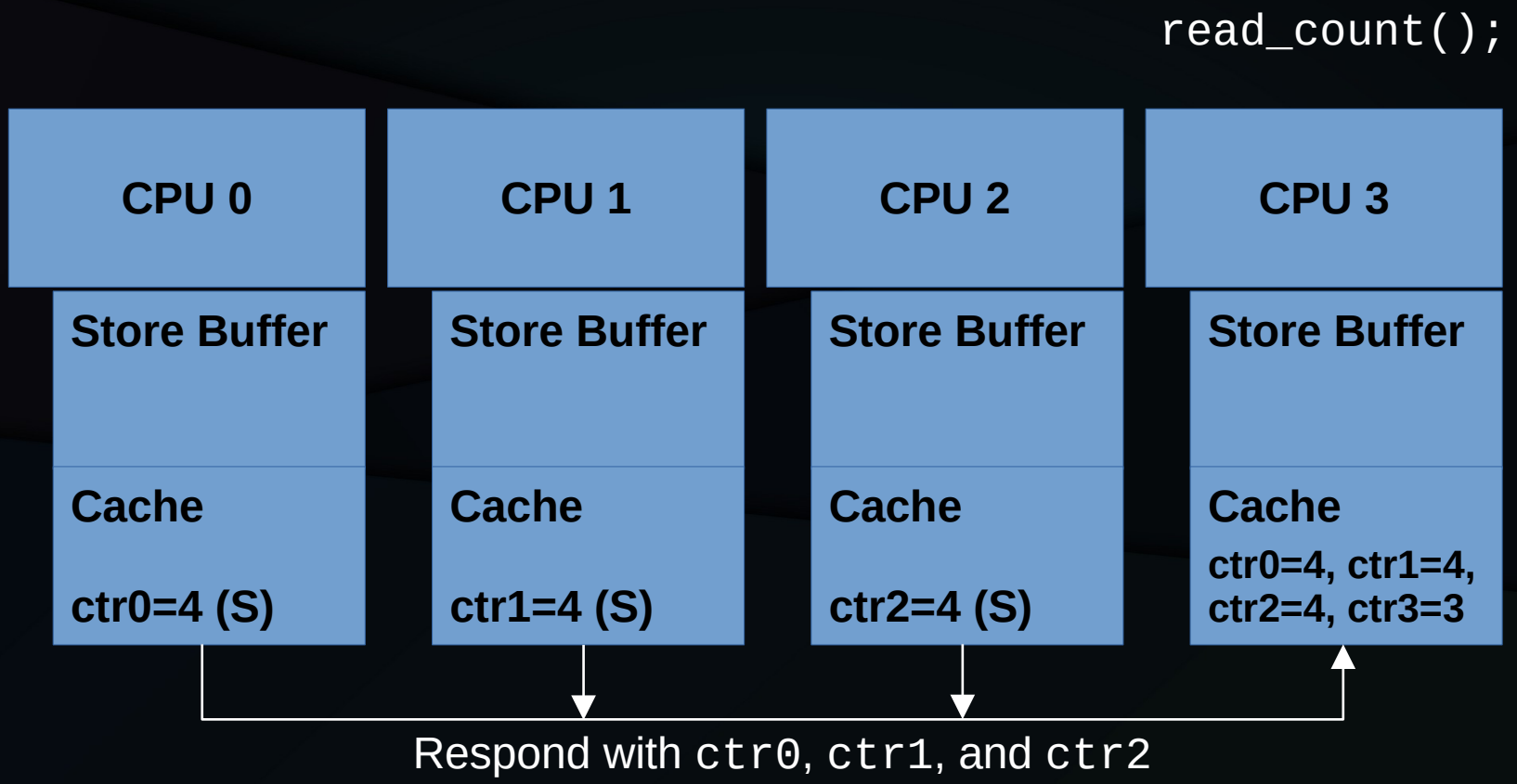
How to Count Better??? (4/7)



How to Count Better??? (5/7)

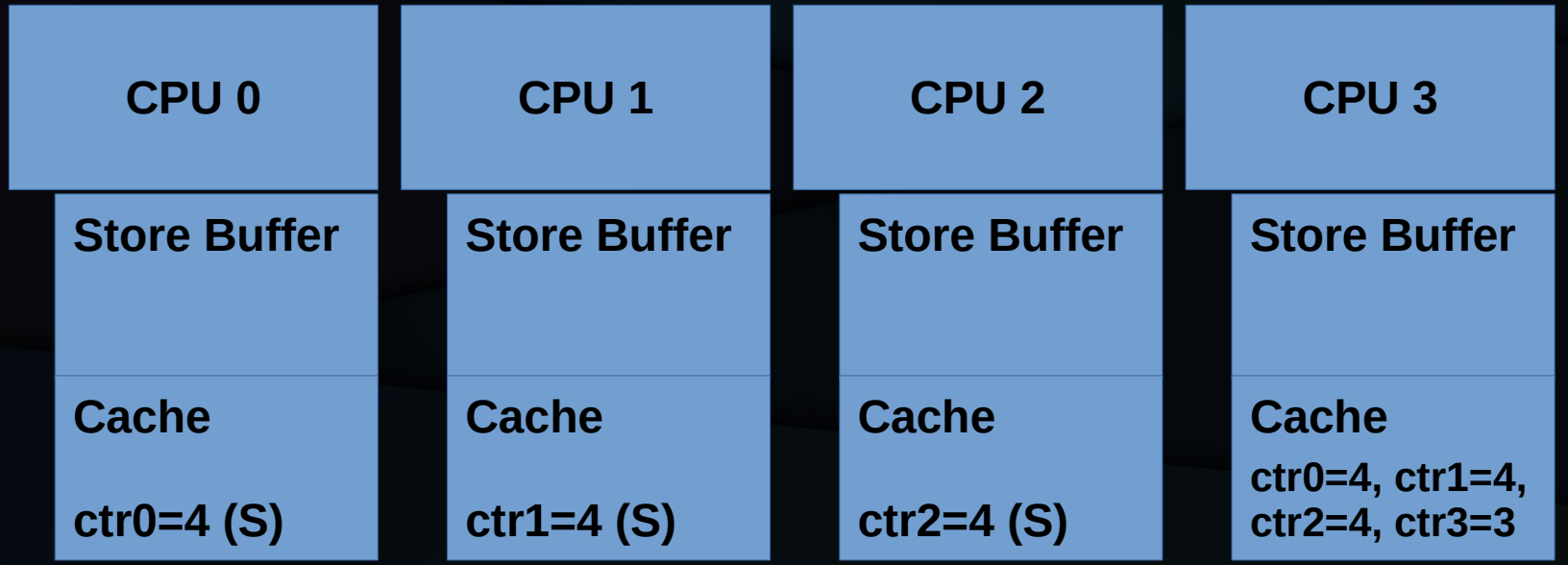


How to Count Better??? (6/7)



How to Count Better??? (7/7)

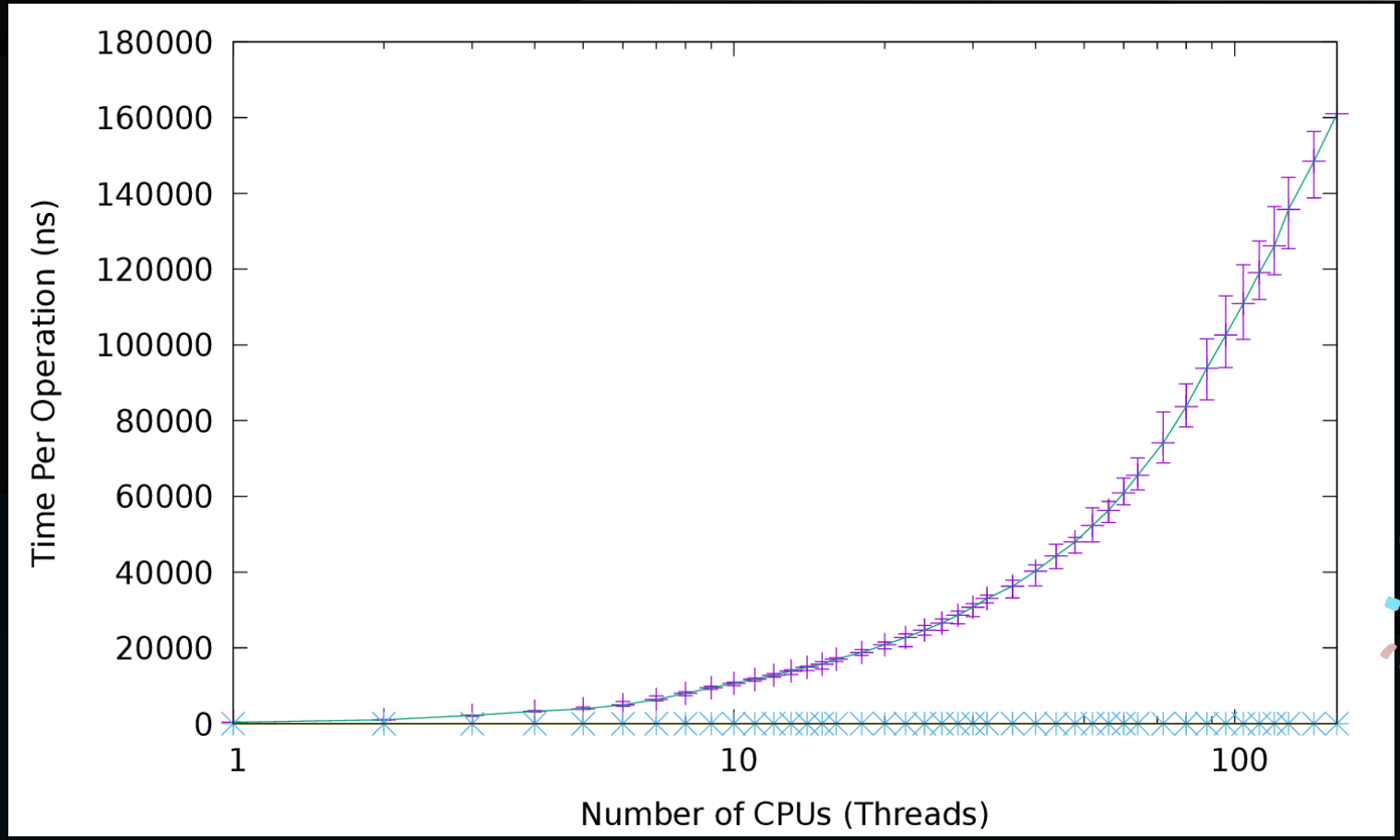
`read_count(); (15)`



How to Count Better Assessment

- Updates are very fast: Non-atomic increment of a local counter variable with no ordering
- Reads are very slow: Sum up ***all*** threads' counters and return this sum
  - Maximum error limited by the potential change in platonic counter value during summation

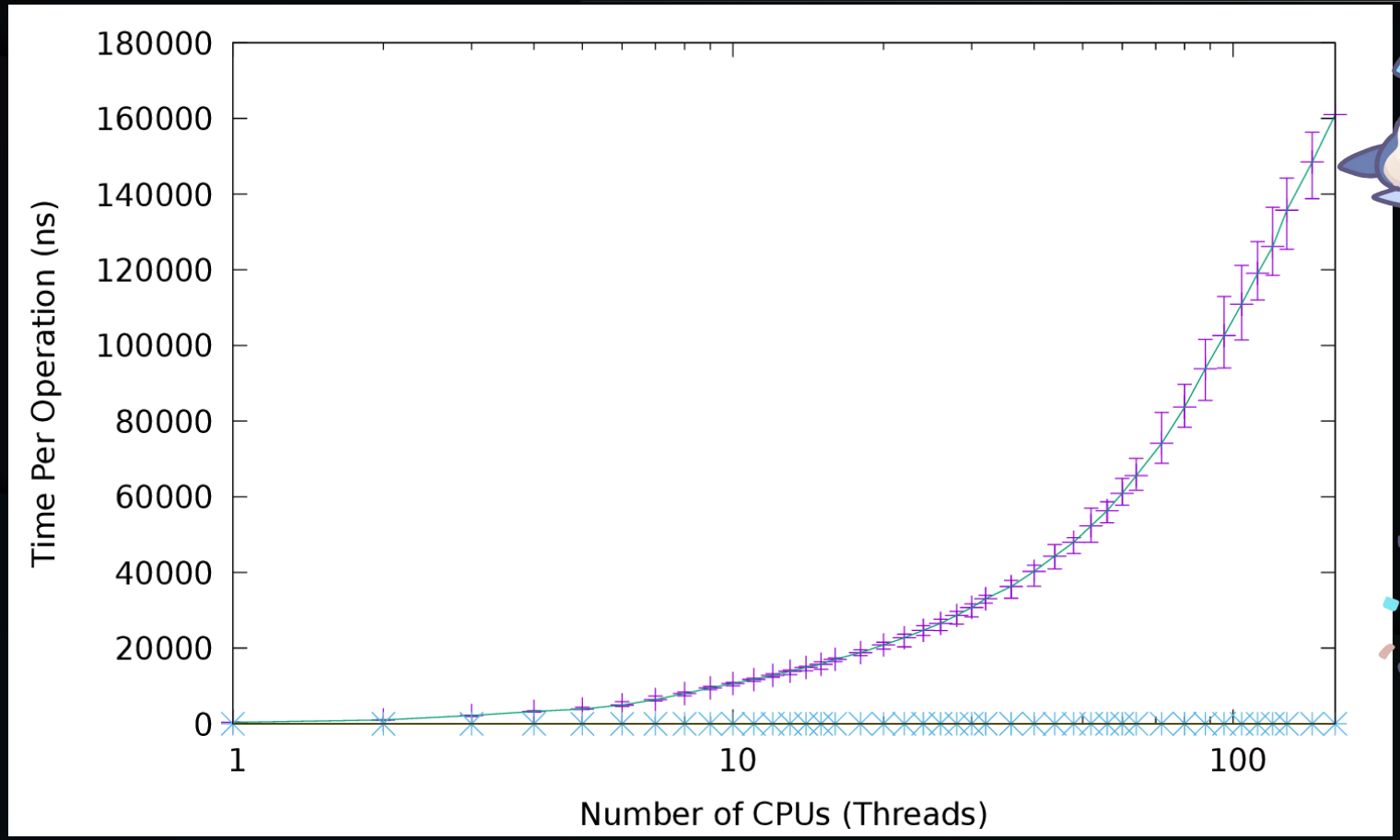
Updates: Accuracy & Scalability!





166-CPU AMD EPYC-Milan Processor @ 2.0GHz

Reads: Not So Much...







166-CPU AMD EPYC-Milan Processor @ 2.0GHz

How to Count Better: Pseudocode

```
unsigned long counter;

static inline void inc_count(void)
{
  WRITE_ONCE(counter, counter + 1);
}

static inline unsigned long read_count(void)
{
  int t;
  unsigned long sum = 0;

  for_each_thread(t)
    sum += READ_ONCE(*counterp[t]);
  return sum;
}
```


For more details: https://mirrors.edge.kernel.org/pub/linux/kernel/people/paulmck/perfbook/perfbook.html Section 5.2

How to Count Better: Pseudocode

```
unsigned long counter;

static inline void inc_count(void)
{
  WRITE_ONCE(counter, counter + 1);
}

static inline unsigned long read_count(void)
{
  int t;
  unsigned long sum = 0;

  for_each_thread(t)
    sum += READ_ONCE(*counterp[t]);
  return sum;
}
```

Updates a few ns, reads 100s of µs!

How to Count Better: Pseudocode

```
unsigned long counter;

static inline void inc_count(void)
{
  WRITE_ONCE(counter, counter + 1);
}

static inline unsigned long read_count(void)
{
  int t;
  unsigned long sum = 0;

  for_each_thread(t)
    sum += READ_ONCE(*counterp[t]);
  return sum;
}
```

But can we read even faster???

Updates a few ns, reads 100s of µs!

Speed-Reading How-to: Pseudocode

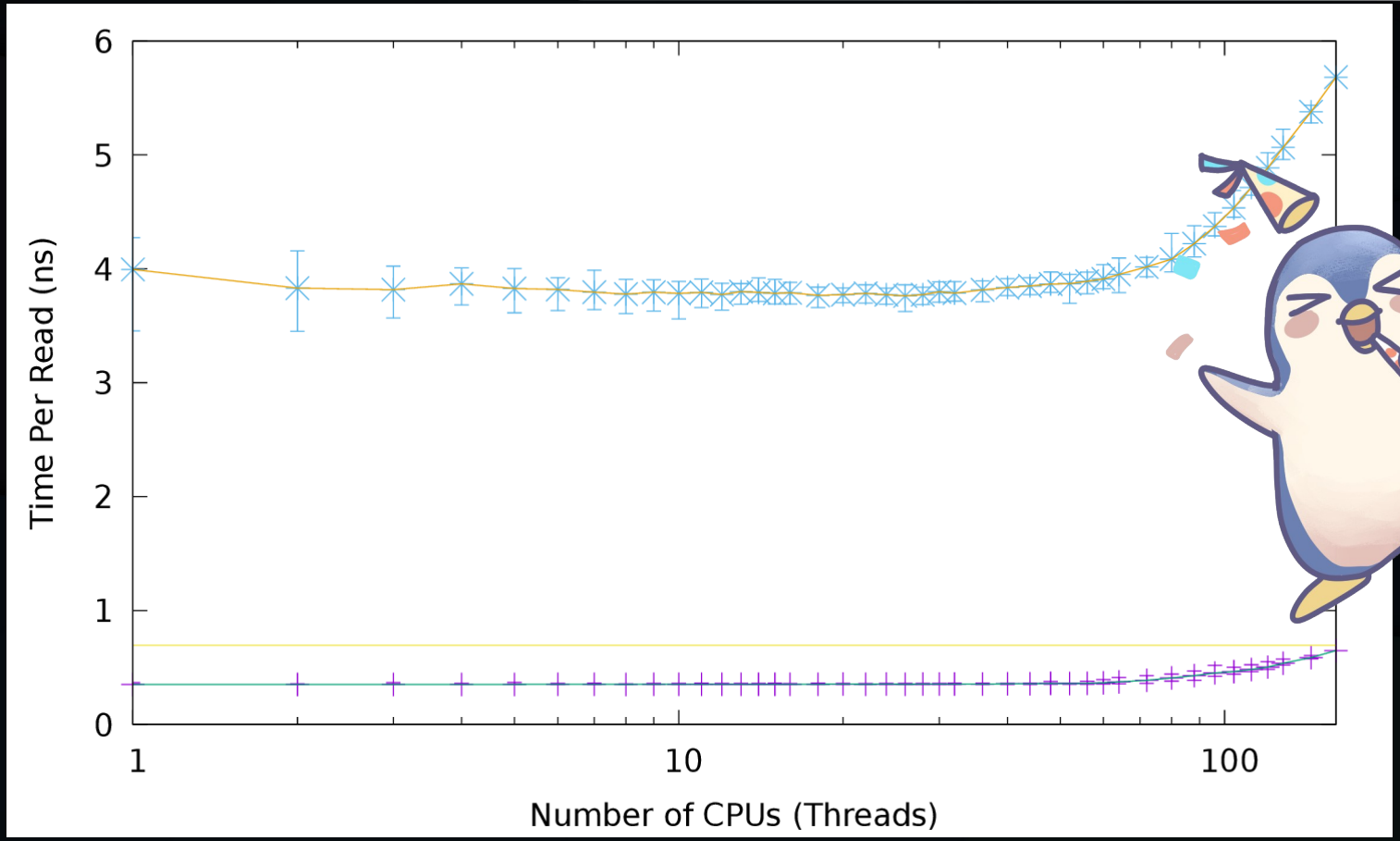
```
unsigned long counter_sum;

static inline unsigned long read_count_fast(void)
{
  return READ_ONCE(counter_sum);
}

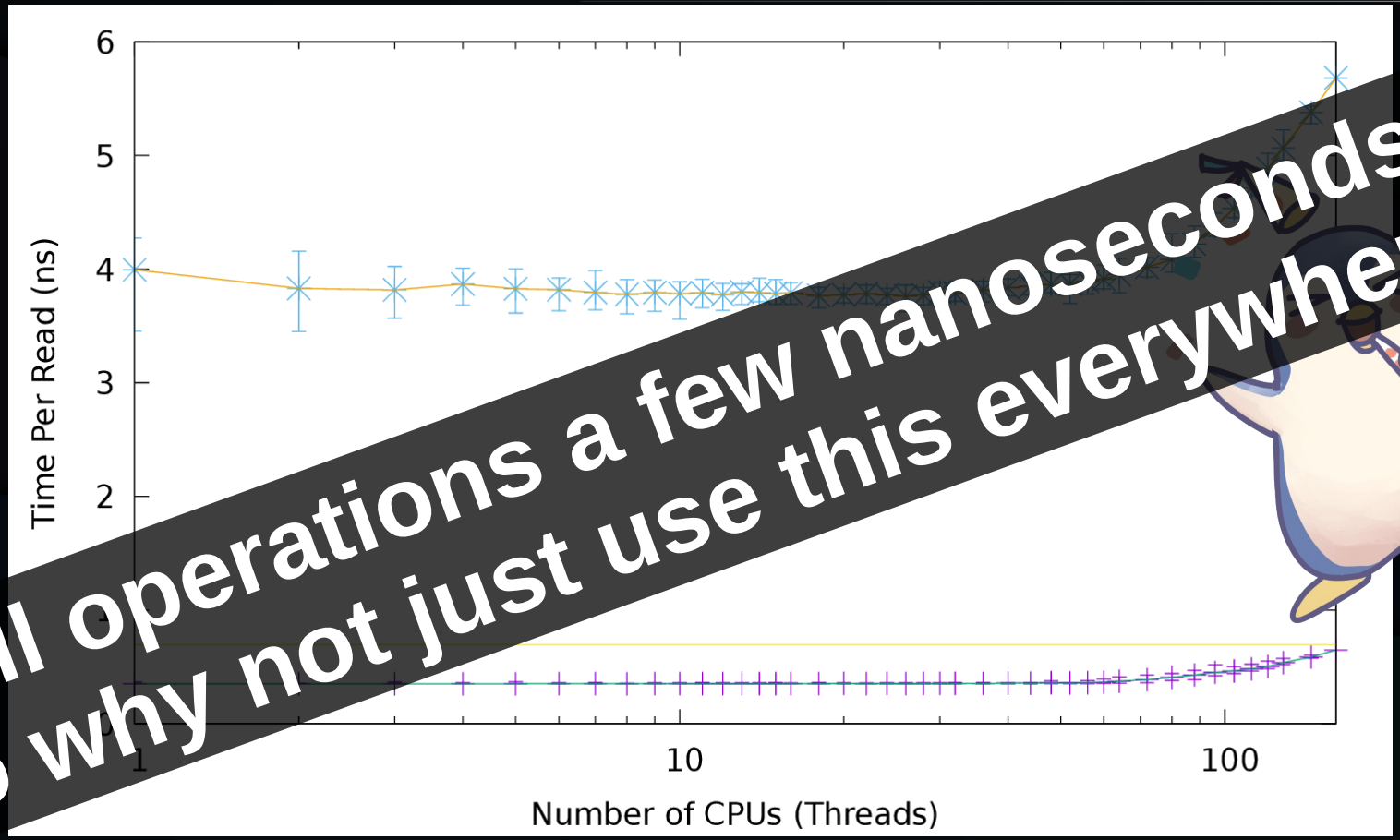
void update_counter_sum(void)
{
  for (;;) {
    WRITE_ONCE(counter_sum, read_count()); // Apply many updates: Batch!
    schedule_timeout_idle(10);             // Sleep 10 jiffies: Delay!
  }
}
```


For more details: https://mirrors.edge.kernel.org/pub/linux/kernel/people/paulmck/perfbook/perfbook.html Section 5.2.4

Accuracy & Scalability for Both!!!



Accuracy & Scalability for Both!!!



All operations a few nanoseconds,
so why not just use this everywhere?



Speed-Reading How-to: Tradeoffs

- Need an extra thread (per counter?)
- That threads's wakeups not good for batteries
- Extra delay from update to read
- Counter reads are normally very infrequent, so why bother?
  - Few will care about 160µs every five seconds...

For more details: https://mirrors.edge.kernel.org/pub/linux/kernel/people/paulmck/perfbook/perfbook.html Section 5.2.4

Other Complications

- Approximate limits (Section 5.3)
- Exact limits (Section 5.4)
  - Without atomic operations (Section 5.4.3)
  - Only sometimes (Section 5.4.6)
- Validation (Section 5.5.1)

Linux-Kernel Case Studies

Statistical Counters (Case 1/3)

Statistical Counters (Case 1/3)

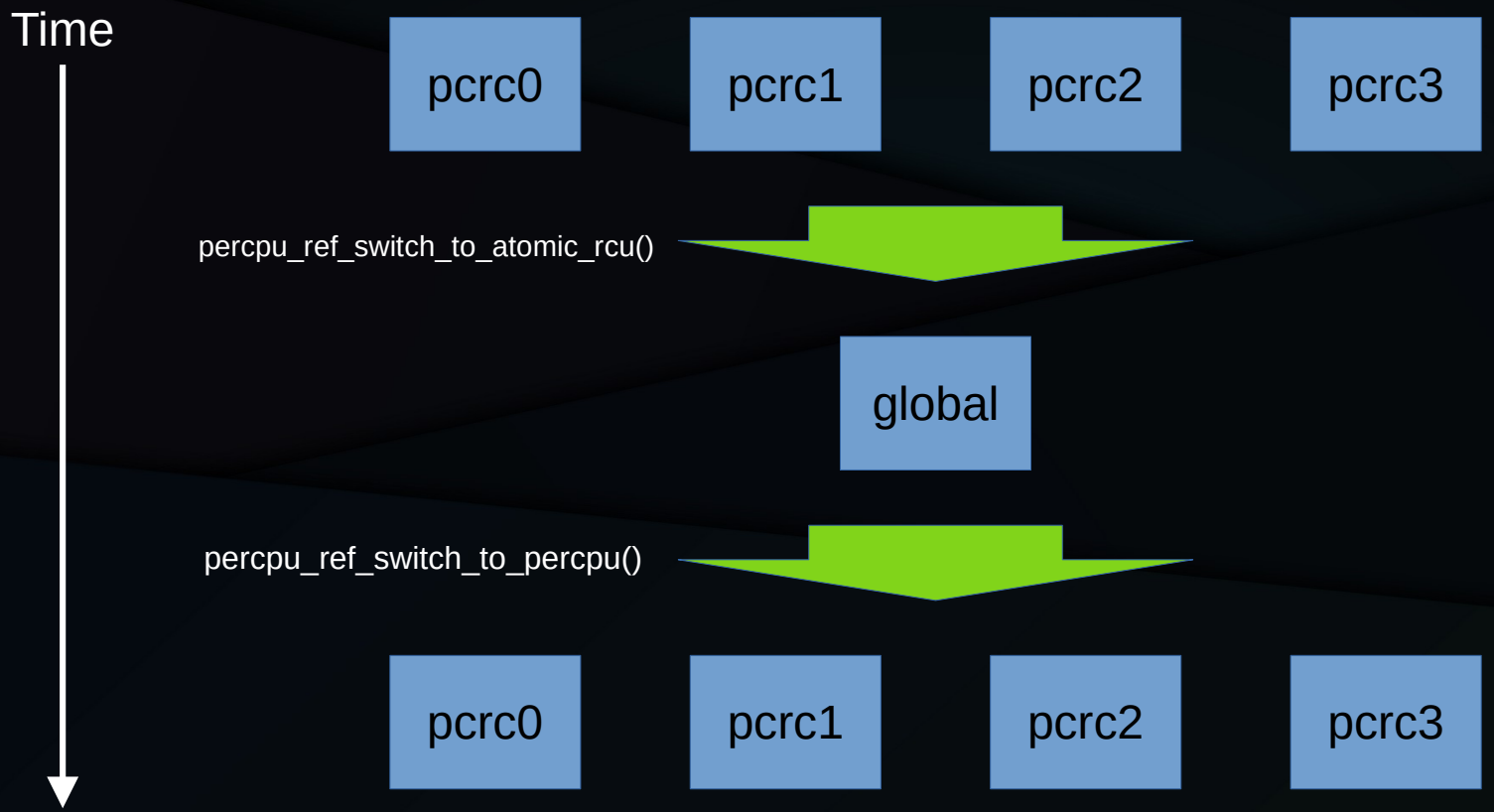
- Heavily used: More than 300 uses of `this_cpu_inc()` and `__this_cpu_inc()`
- Additional uses as throughput counters
  - For example `this_cpu_inc()` to count network packets and `this_cpu_add()` to count bytes
- Typically open-coded
- Taken for granted since at least 1987!

Per-CPU Refcounts (Case 2/3)

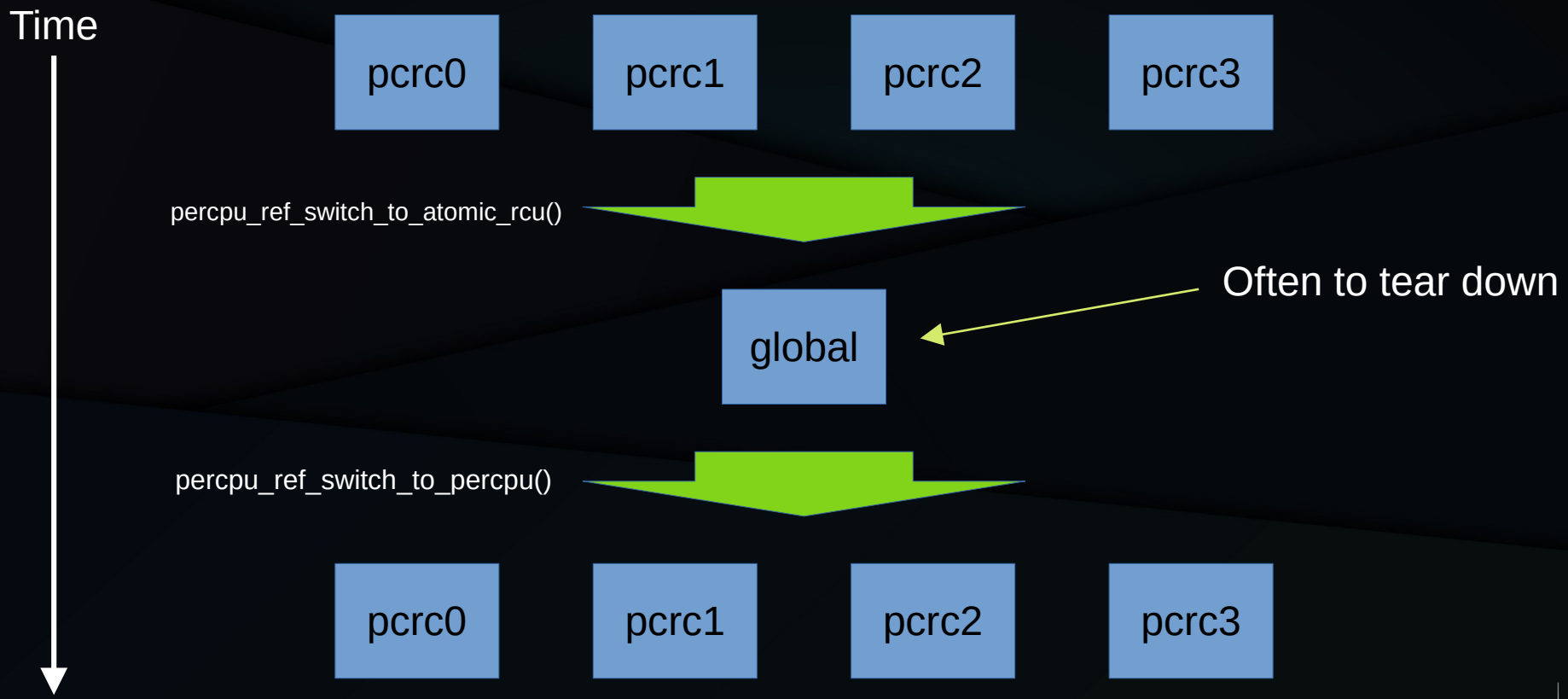
Per-CPU Refcounts (Case 2/3)

- Switch from per-CPU to global mode:
  - Start with counter equal to 1
  - Per-CPU refcounts used in common case
  - At tear-down time, switch to global refcount
    - Combine sum of per-CPU refcounts to global
    - If tearing down, wait for RCU grace period
  - Remove initial reference
- Can also switch back to per-CPU if not tearing down

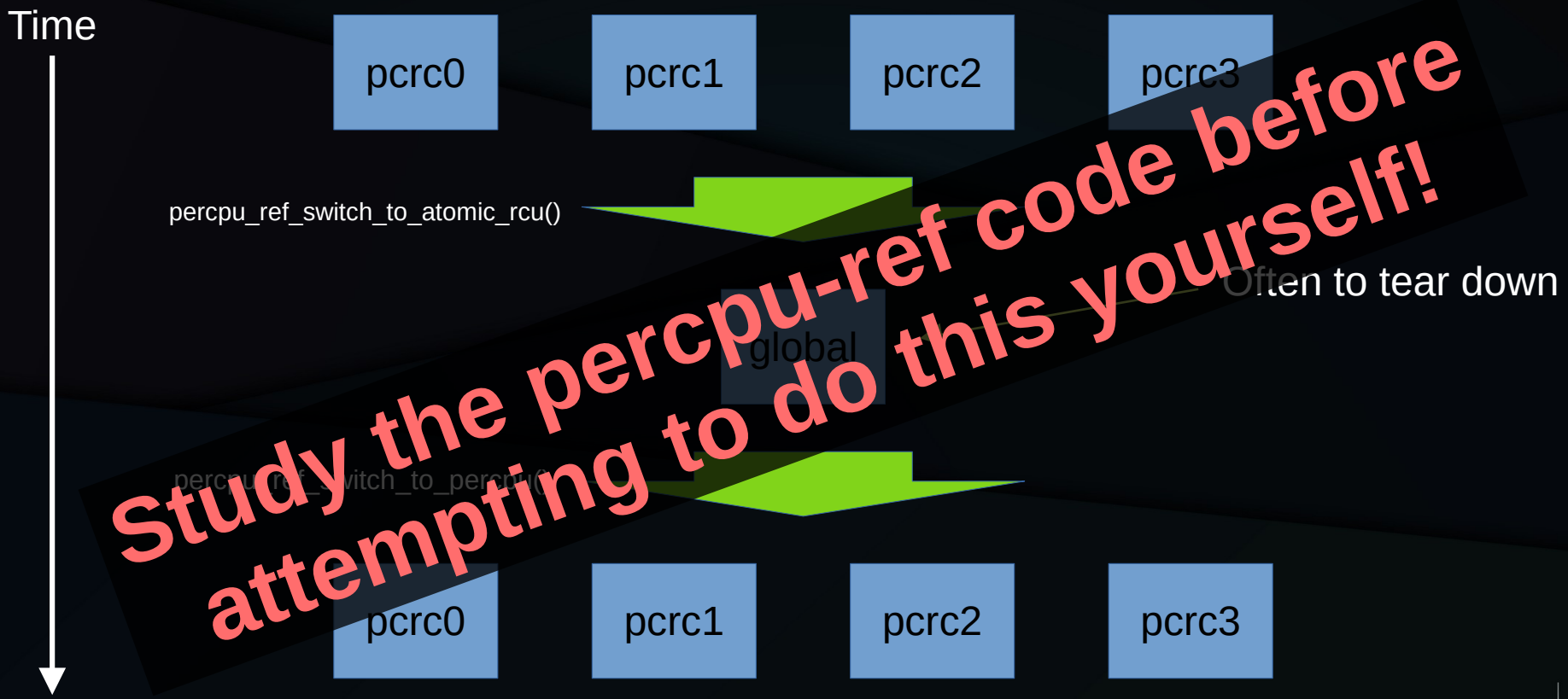
Per-CPU Refcounts (Case 2/3)



Per-CPU Refcounts (Case 2/3)



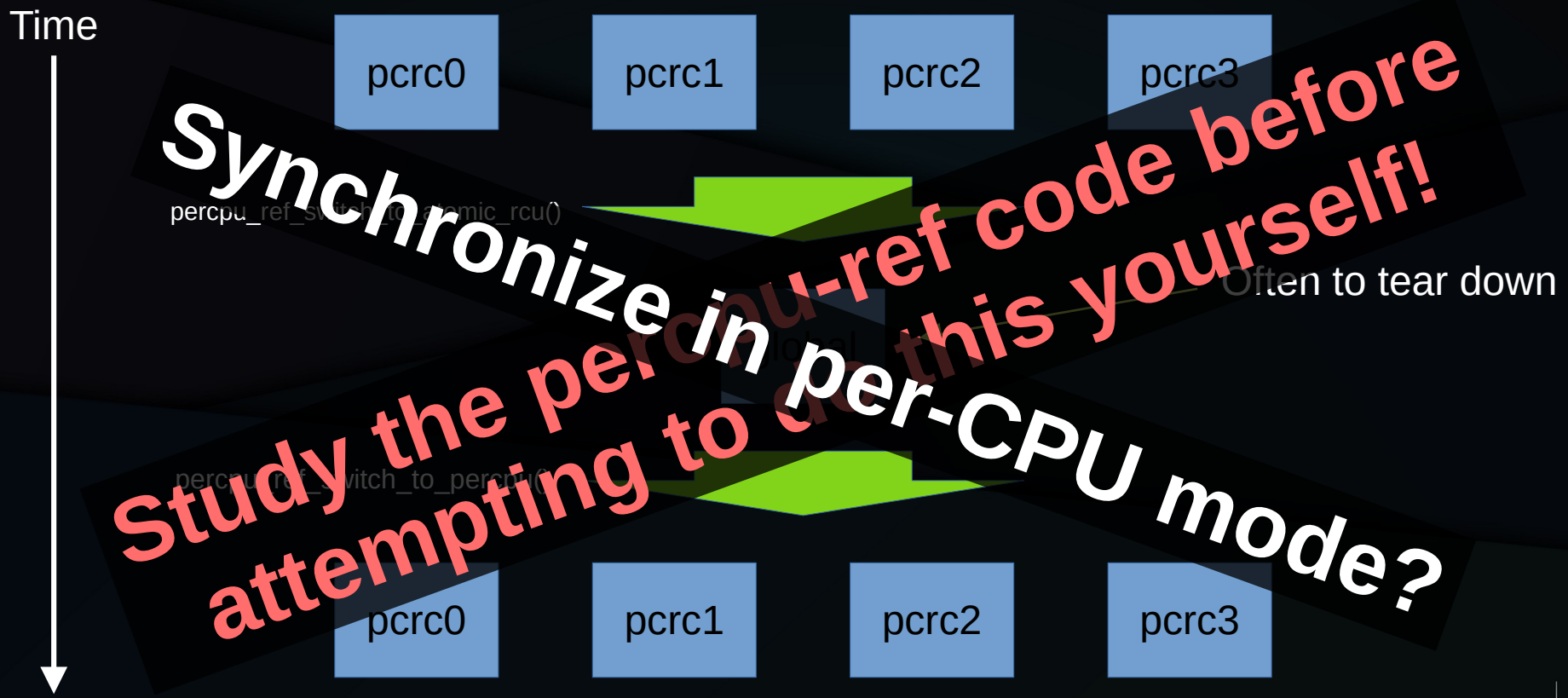
Per-CPU Refcounts (Case 2/3)



Often to tear down

Study the percpu-ref code before attempting to do this yourself!

Per-CPU Refcounts (Case 2/3)

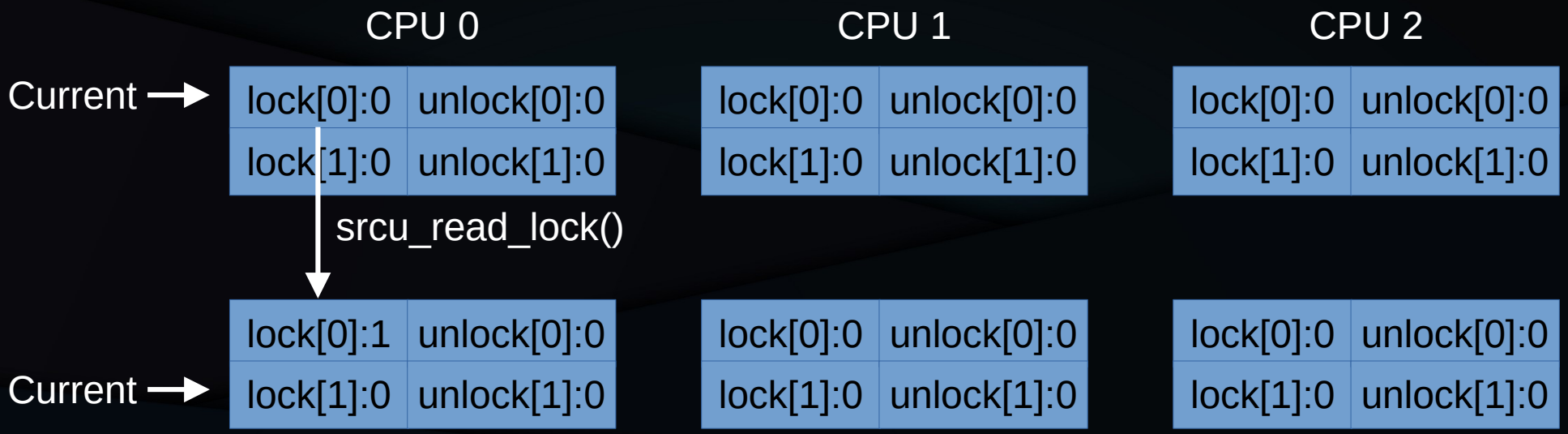


SRCU Counts of Readers (Case 3/3)

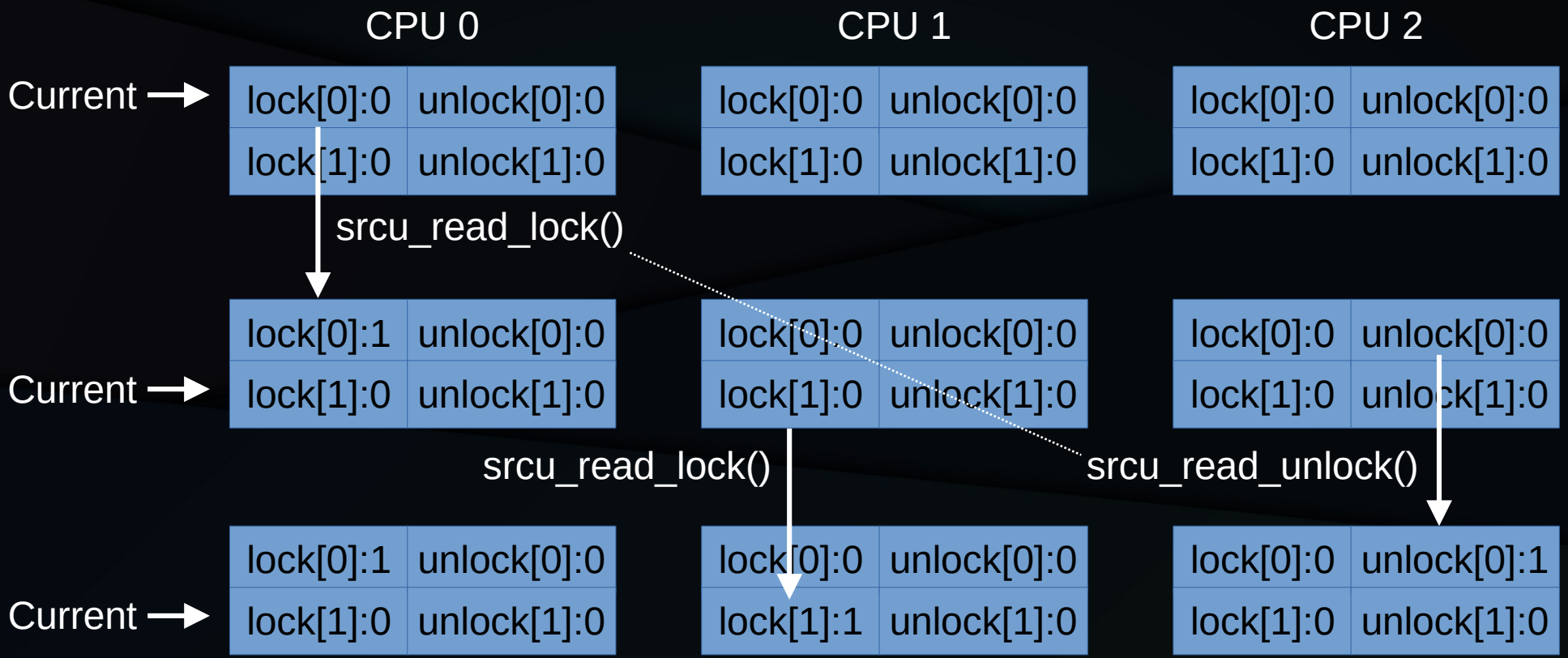
SRCU Counts of Readers (Case 3/3)

	CPU 0		CPU 1		CPU 2	
Current →	lock[0]:0	unlock[0]:0	lock[0]:0	unlock[0]:0	lock[0]:0	unlock[0]:0
	lock[1]:0	unlock[1]:0	lock[1]:0	unlock[1]:0	lock[1]:0	unlock[1]:0

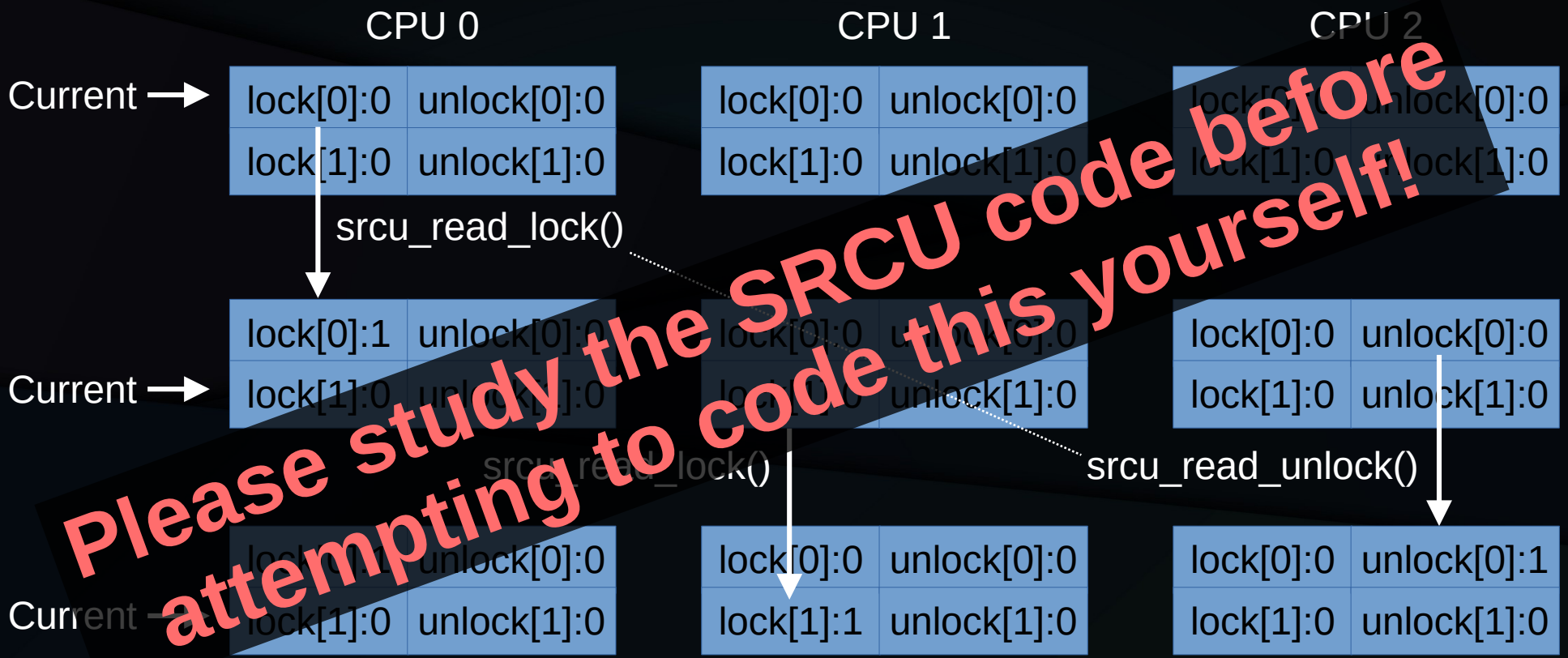
SRCU Counts of Readers (Case 3/3)



SRCU Counts of Readers (Case 3/3)



SRCU Counts of Readers (Case 3/3)



Counting’s General Lessons

Counting's General Lessons

- Partitioning helps performance and scalability
- Partial partioning helps, at least partially
- Batching updates can also help
- Careful use of delay can help greatly (Chapter 9)
- Engineering, not science: Thus tradeoffs
  - Hardware and workloads affect design

Pratique du Comptage

Summary

Summary

- Modern hardware is highly optimized
  - Most of the time!
  - Incremental improvements due to integration
  - But the speed of light is too slow and atoms too big
- Use concurrent software where available
- Structure your code & data to avoid big obstacles
  - Plenty of tricks for fast concurrent counting!

For More Information

- “Is Parallel Programming Hard, And, If So, What Can You Do About It?”
  - https://mirrors.edge.kernel.org/pub/linux/kernel/people/paulmck/perfbook/perfbook.html
  - Chapter 3 (“Hardware and its Habits”) and Chapter 5 (“Counting”)
- “What every programmer should know about memory, Part 1”
  - https://lwn.net/Articles/250967/ (contains links to parts 2-9)
- “Who's afraid of a big bad optimizing compiler?”
  - https://lwn.net/Articles/793253/
- “Calibrating your fear of big bad optimizing compilers”
  - https://lwn.net/Articles/799218/
- “What Every C Programmer Should Know About Undefined Behavior #3/3”
  - https://blog.llvm.org/2011/05/what-every-c-programmer-should-know_21.html (links to #1 & #2)

L'antidote de Codage Simultané de la Femme de Paul

- Cordial de mûres sauvages de l'Himalaya
  - Mettre deux litres de mûres dans un pot de quatre litres
  - Ajouter cinq huitièmes litres de sucre
  - Remplissez le pot de vodka
  - Secouez tous les jours pendant cinq jours
  - Secouez chaque semaine pendant cinq semaines
  - Passer au tamis: Ajoutez des baies à la glace, consommez le liquide filtré comme vous voulez

Paul’s Wife’s Concurrency Antidote

- Wild Himalayan Blackberry Cordial
  - Put 8 cups wild himalayan blackberries in 1 gallon jar
  - Add 2½ cups sugar
  - Fill jar with vodka
  - Shake every day for five days
  - Shake every week for five weeks
  - Pour through sieve: Add berries to ice cream, consume filtered liquid as you wish

Paul’s Wife’s Concurrency Antidote

- Wild Himalayan Blackberry Cordial
  - Put 8 cups wild himalayan blackberries in 1 gallon jar
  - Add 2½ cups sugar
  - Fill jar with vodka
  - Shake every day for five days
  - Shake every week for five weeks
  - Pour through sieve: Add berries to ice cream, consume filtered liquid as you wish

But what constitutes an overdose???

Recette de Test de Surdosage

- 1 liter de chapelure de pain sec
- 100 ml. de sauge, 60 ml. d’oignons, 100 ml. de céleri
- 100 ml. de maïs soufflé (non cuit), une pincée de sel
- 1 liter de bouillon

Bien mixer, farcir la dinde et cuire 5 heures à 150°C ou jusqu'à ce que le maïs soufflé fasse exploser la dinde.

Overdose-Test Recipe

- 4 c. crushed dry bread
- ½ c. sage, ¼ c. onions, ½ c. celery
- ½ c. uncooked popcorn, 1 tsp. salt
- 5 c. broth

Mix well; stuff turkey. Cook 5 hours at 300°F or until popcorn blows the a** off the turkey.

Overdose-Test Recipe

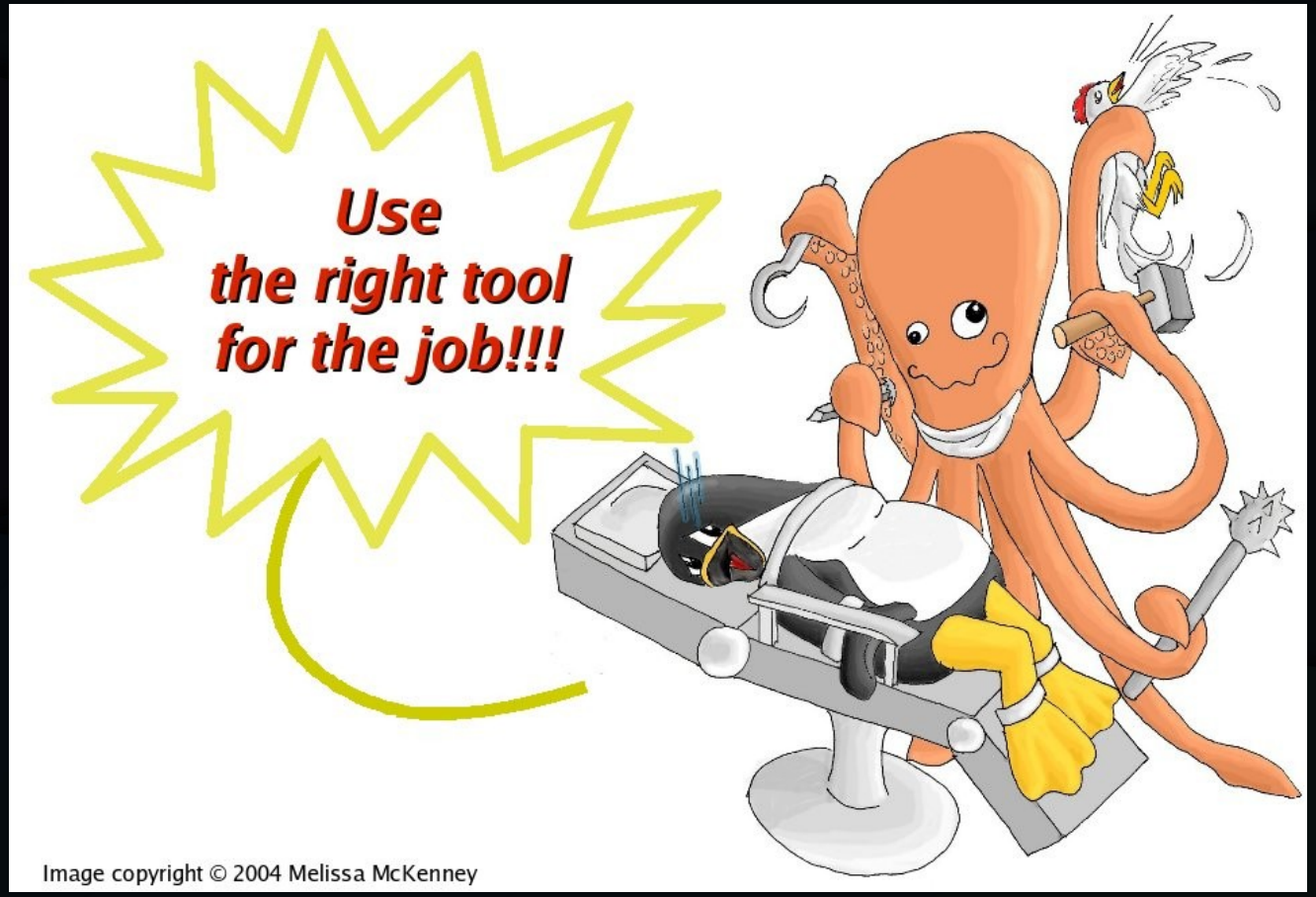
- 4 c. crushed dry bread
- ½ c. sage, ¼ c. onions, ½ c. celery
- ½ c. uncooked popcorn, 1 tsp. salt
- 5 c. broth

Mix well, stuff turkey. Cook 5 hours at 300°F or until popcorn blows the a** off the turkey.

If this recipe seems like a good idea, you have well and truly overdosed!!!

And As Always...

And As Always...



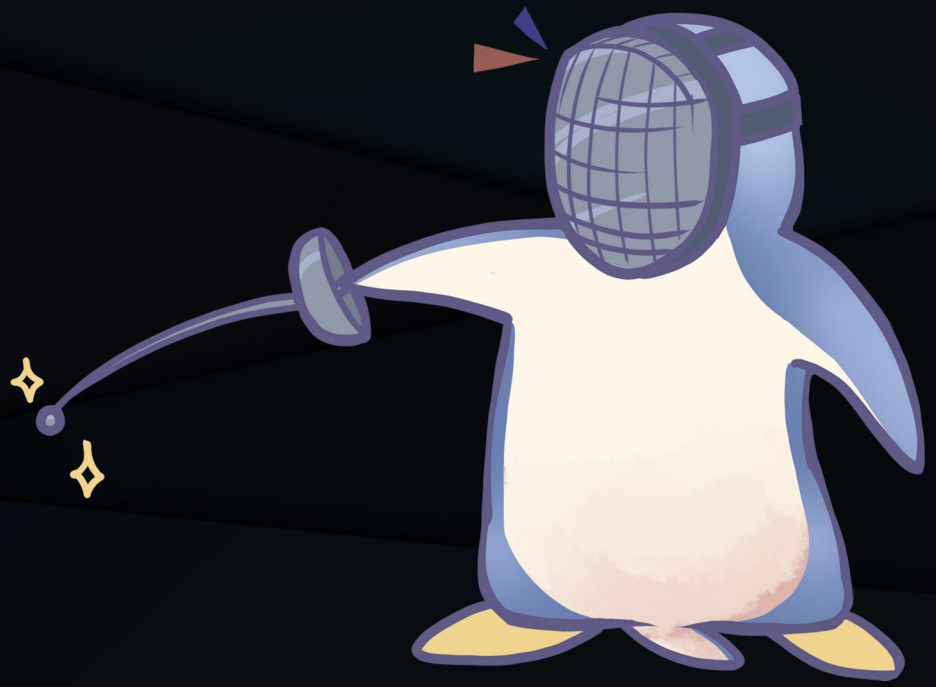
If there is no right tool, invent it!!!

Questions?



https://mirrors.edge.kernel.org/pub/linux/kernel/people/paulmck/perfbook/perfbook.html Chapter 5

Backup



Who is Afraid of an Optimizer???

Who is Afraid of an Optimizer???

- If you are wise, you will be! Compilers can:
  - Reorder references
  - Fuse and merge loads and stores
  - Tear loads and stores
  - Invent loads and stores
  - Omit “unused” loads and stores
- Which can fatally surprise your concurrent code

https://lwn.net/Articles/793253/ “Who's afraid of a big bad optimizing compiler?”
https://lwn.net/Articles/799218/ ”Calibrating your fear of big bad optimizing compilers”

Who is Afraid of an Optimizer???

- And don’t forget undefined behavior! Compilers can:
  - Assume signed ints never wrap (except in the kernel!)
  - Treat non-atomic non-volatile variables as private
    - See previous slide
  - Assume array indexes are always in-bounds
  - Assume pointers from two different kmalloc() calls are never equal to each other
- Which can also fatally surprise your concurrent code

https://lwn.net/Articles/793253/ “Who's afraid of a big bad optimizing compiler?”
https://lwn.net/Articles/799218/ ”Calibrating your fear of big bad optimizing compilers”

PDP-12 and Trigonometric Functions

Paul’s and Bryan’s 1977 Schoolwork



DEC PDP-12

625 KHz CPU (1.6µs/ref)
One register (why more?)
12-bit instructions 1.6µs
4Kx12 bits core memory
No stack & no caches
512x512 graphics (green!)
No graphics frame buffer
Size of a refrigerator
sin() & cos() < 21µs

Photo in museum

“And we were #$(*&@# happy to have 4Kx12 bits!!!”

https://www.rcsri.org/collection/pdp-12/skylark-ps.txt
Photo: The Retro-Computing Society of Rhode Island, Inc.

sin() & cos() < 21µs? How???

- Forget degrees and radians
- Divide the circle into 128 parts
- Use a lookup table

sin() & cos() < 21µs? Here's How!!!

```
TRGMSK, 0177            / Variable named TRGMSK with value 0177
TRGSTO, 0               / Temporary variable
SINSRC, 0
        AND TRGMSK      / 3.2µs
        TAD SNTABL      / 3.2µs
        DCA TRGSTO      / 3.2µs
        TAD I TRGSTO    / 4.8µs
        JMP I SINSRC    / 3.2µs
                        /20.8µs Total (24.0µs including JMS)
```

sin() & cos() < 21µs? Here's How!!!

```
SINTBL, 0000            /0 Degrees, values octal s.bbbbbbbbbbb
        0144
        0310
        0454
        0617
        0761
        1122
        1261
        1417 ...
```